Laporan Proyek Akhir
Sistem Inventori Barang

Disusun oleh:

Fadhil Afiq Badruzzaman (20240801119)
Dafa Rafi Nur Wansyhah (20240801119)
Luqman Syahreno (20240801016)
Rizqi Bagas Wicaksono (20240801187)
Angga Aditya (20240801165)

Dosen pengampu:
DWI SARTIKA SIMATUPANG , S.T., M.T.I.

FAKULTAS ILMU KOMPUTER
PRODI TEKNIK INFORMATIKA
UNIVERSITAS ESA UNGGUL
202

DAFTAR ISI

DAFTAR ISI ii
Daftar Gambar iv
Daftar Tabel iv
BAB I PENDAHULUAN 1
1.1 Tujuan Penulisan Dokumen 1
1.2 Lingkup Masalah 1
1.3 Definisi, Akronim, dan Singkatan 2
1.4 Referensi 3
1.5 Struktur Dokumen 3
BAB II DESKRIPSI GLOBAL 4
2.1 Perspektif Produk 4
2.2 Fungsi Produk 5
2.3 Karakteristik Pengguna 5
2.4 Batasan 6
2.5 Asumsi dan Ketergantungan 7
BAB III KEBUTUHAN SPESIFIK 8
3.1 Kebutuhan Antarmuka Eksternal 8
3.1.1 Antarmuka Pengguna (*User Interface*) 8
3.1.2 Struktur Navigasi 8
3.1.3 Mekanisme Pesan Sistem (*System Feedback*) 9
3.2 Kebutuhan Fungsional 9
3.3 Deskripsi Data 11
3.4 Kebutuhan Non-Fungsional 12
3.5 Atribut Kualitas (ISO/IEC 25010) 13
3.6 Tools dan Teknologi yang Digunakan 14
3.6.1 Bahasa Pemrograman & Framework 15
3.6.2 Basis Data (*Database*) 15
3.6.3 Perangkat Lunak Pengembangan (*Development Tools*) 15
BAB IV DESKRIPSI PROSES DAN ALIRAN DATA 16
4.1 Diagram Konteks (Level 0) 16

4.1.1 Interaksi Entitas Admin....16
4.1.2 Interaksi Entitas User....17
4.1.3 Interaksi dengan Penyimpanan Data (*Data Store*)....17
4.2 *Data Flow Diagram* (DFD) Level 1....17
4.2.1 Proses 1.1: Kelola Data Barang....18
4.2.2 Proses 1.2: Kelola Transaksi Barang....18
4.3 *Use Case Diagram* dan Deskripsi....19
4.3.1 Identifikasi Aktor....21
4.3.2 Definisi Use Case....21
4.4 Deskripsi Teknis Sistem....22
4.5 Perancangan Antarmuka....22
BAB V IMPLEMENTASI SISTEM....28
5.1 Tujuan Implementasi....28
5.2 Lingkungan Pengembangan....28
5.3 Struktur Direktori Aplikasi....29
5.4 Implementasi Fitur Utama....31
5.5 Keamanan dan Kontrol Akses....39
5.5.1 Pengamanan Kredensial (*Password Hashing*)....39
5.5.2 Otorisasi Berbasis Peran (*Role-Based Access Control*)....39
BAB VI PENGUJIAN SISTEM....40
6.1 Pengujian *Black Box*....40
6.2 Skenario Pengujian....41
BAB VII PEMELIHARAAN SISTEM....43
7.1 Pemeliharaan Korektif (*Corrective Maintenance*)....43
7.2 Pemeliharaan Adaptif (*Adaptive Maintenance*)....44
7.3 Pemeliharaan Preventif (*Preventive Maintenance*)....44
7.4 Pemeliharaan Sempurna (*Perfective Maintenance*)....44
KESIMPULAN....45

Daftar Gambar

Gambar 1 ERD....11
Gambar 2 DFD Level 0....16
Gambar 3 DFD Level 1....17
Gambar 4 Usecase....20
Gambar 5 Desain halaman login....22
Gambar 6 Desain halaman dashboard....23
Gambar 7 Desain dalaman inventori....24
Gambar 8 Desain halaman produk....25
Gambar 9 Desain halaman laporan....26
Gambar 10 Desain halaman peringatan....27
Gambar 11 Desain halaman pengaturan....27
Gambar 12 Struktur Folder....30
Gambar 13 Halaman Login....33
Gambar 14 Halaman dashboard....34
Gambar 15 Halaman inventori....35
Gambar 16 Halaman produk....36
Gambar 17 Halaman laporan....37
Gambar 18 Halaman peringatan....38
Gambar 19 Halaman pengaturan....38

Daftar Tabel

Table 1 Rincian kebutuhan fungsional....9
Table 2 Rincian kebutuhan non-fungsional....12
Table 3 Spesifkasi perangkat lunak dan alat bantu....29
Table 4 Pengujian....41

BAB I
PENDAHULUAN

1.1 Tujuan Penulisan Dokumen

Pengelolaan data inventori yang akurat dan efisien merupakan elemen krusial dalam menjaga kelancaran operasional suatu gudang atau organisasi. Namun, metode pencatatan inventori yang masih dilakukan secara manual atau konvensional sering kali memiliki keterbatasan signifikan. Ketergantungan pada pencatatan manual rentan terhadap risiko kesalahan manusia (*human error*), yang mengakibatkan selisih antara jumlah fisik barang dengan data administrasi, serta keterlambatan dalam penyajian laporan stok. Ketidakakuratan ini dapat menghambat proses pengambilan keputusan dan menurunkan efisiensi pengelolaan sumber daya.

Sebagai solusi atas permasalahan tersebut, dalam proyek akhir ini dikembangkanlah **'Sistem Inventori Barang'** berbasis web. Pengembangan sistem ini ditujukan untuk mentransformasi proses manual menjadi sistem digital yang terintegrasi, yang mampu menangani pencatatan transaksi barang masuk dan barang keluar secara otomatis serta memperbarui status stok secara *real-time*. Dengan memanfaatkan teknologi web, sistem ini dirancang untuk memberikan kemudahan akses, kecepatan pemrosesan data, dan jaminan integritas data yang lebih baik dibandingkan metode konvensional.

Dokumen Laporan Proyek Akhir ini disusun dengan tujuan utama untuk mendokumentasikan seluruh rangkaian siklus hidup pengembangan perangkat lunak (*Software Development Life Cycle*) yang telah dilaksanakan. Dokumentasi ini mencakup tahapan **analisis kebutuhan** spesifik sistem, **perancangan arsitektur** dan basis data, serta proses **implementasi kode** (*coding*) yang dibangun menggunakan teknologi Next.js dan PostgreSQL. Lebih lanjut, laporan ini juga memaparkan hasil **pengujian sistem** yang dilakukan melalui metode *Black Box Testing* untuk memvalidasi fungsionalitas fitur, serta merinci strategi **pemeliharaan sistem** (*maintenance*) guna menjamin stabilitas dan keberlanjutan perangkat lunak dalam jangka panjang.

1.2 Lingkup Masalah

Agar pengembangan sistem tetap terarah dan sesuai dengan tujuan yang telah ditetapkan, penulis membatasi ruang lingkup masalah pada aspek-aspek berikut:

1. **Fokus Fungsional Sistem** Pengembangan sistem difokuskan secara eksklusif pada **manajemen inventori internal** untuk memantau pergerakan fisik barang di gudang. Sistem ini **tidak** mencakup fitur penjualan (*Point of Sales*/POS), penetapan harga jual, maupun integrasi dengan sistem akuntansi keuangan. Dengan demikian, sistem tidak menangani kalkulasi profit atau neraca keuangan, melainkan berfokus pada keakuratan data kuantitas barang.

2. **Cakupan Fitur Utama** Sesuai dengan analisis kebutuhan fungsional, fitur-fitur yang diimplementasikan dalam sistem meliputi:
    - **Otentikasi:** Mekanisme *Login* untuk memverifikasi hak akses pengguna sebelum memasuki sistem.
    - **Manajemen Data Master:** Pengelolaan data induk barang (tambah, ubah, hapus data atribut barang) yang menjadi referensi utama.
    - **Transaksi Inventori:** Pencatatan **Barang Masuk** yang secara otomatis menambah jumlah stok, dan **Barang Keluar** yang mengurangi jumlah stok dengan validasi ketersediaan barang.
    - **Pelaporan:** Penyajian Laporan Stok terkini dan rekapitulasi riwayat transaksi untuk periode waktu tertentu.

3. **Aktor Pengguna** Sistem dirancang untuk digunakan oleh dua jenis aktor dengan hak akses yang berbeda, yaitu:
    - **Admin:** Pengguna dengan hak akses penuh untuk mengelola data master barang, manajemen data pengguna, dan melihat laporan.
    - **Petugas Gudang (*User*):** Pengguna operasional yang bertugas mencatat transaksi harian (barang masuk dan keluar) serta memantau ketersediaan stok.

4. **Platform Teknologi** Sistem Inventori Barang dibangun sebagai aplikasi **berbasis web** (*web-based application*). Aplikasi ini dirancang agar dapat diakses melalui peramban web (*web browser*) modern tanpa memerlukan instalasi perangkat lunak khusus pada sisi klien.

1.3 Definisi, Akronim, dan Singkatan

Untuk menghindari kesalahpahaman dalam interpretasi istilah yang digunakan di seluruh dokumen ini, berikut adalah daftar definisi dan akronim penting yang berkaitan dengan pengembangan sistem:

- **Sistem Inventori** Suatu rangkaian proses terkomputerisasi yang bertujuan untuk mengelola pencatatan dan pengendalian jumlah barang fisik yang tersedia di dalam gudang. Dalam konteks laporan ini, istilah tersebut merujuk pada aplikasi berbasis web yang dikembangkan untuk menangani data dan transaksi inventori organisasi.

- **Next.js** Kerangka kerja (*framework*) pengembangan web berbasis React yang digunakan untuk membangun aplikasi *fullstack* dalam proyek ini. Teknologi ini memungkinkan penanganan antarmuka pengguna (*frontend*) dan logika sisi server (*backend*) secara terpadu.

- **Black Box Testing** Metode pengujian perangkat lunak yang berfokus sepenuhnya pada persyaratan fungsional aplikasi tanpa melihat struktur kode internal atau logika algoritma yang membangunnya. Dalam metode ini, sistem diperlakukan sebagai "kotak hitam" di mana pengujian hanya mengevaluasi kesesuaian antara *input* yang diberikan dan *output* yang dihasilkan.

- **Admin** Pengguna dengan hak akses penuh (*superuser*) dalam sistem yang bertanggung jawab atas pengelolaan data pengguna, pengelolaan data induk (*master*) barang, serta pengawasan laporan inventori secara keseluruhan.

- **Petugas Gudang (*User*)** Pengguna dengan hak akses terbatas yang bertanggung jawab terhadap operasional harian gudang. Tugas utama aktor ini meliputi pencatatan transaksi barang masuk dan barang keluar, serta pemantauan ketersediaan stok barang melalui sistem.

1.4 Referensi

- Sommerville, Ian. *Software Engineering*. Pearson Education.
- Pressman, Roger S. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.
- IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
- Materi perkuliahan Mata Kuliah Rekayasa Perangkat Lunak – Universitas Esa Unggul.

1.5 Struktur Dokumen

Sistematika penulisan Laporan Proyek Akhir ini disusun secara terstruktur menjadi tujuh bab utama. Hal ini bertujuan untuk memberikan gambaran yang komprehensif mengenai seluruh tahapan rekayasa perangkat lunak yang dilakukan, mulai dari perencanaan hingga strategi pemeliharaan pasca-implementasi. Berikut adalah ringkasan isi dari masing-masing bab:

- **Bab I: Pendahuluan** Bab ini menguraikan latar belakang masalah, tujuan penulisan dokumen, ruang lingkup pengembangan sistem, serta definisi dan istilah teknis yang digunakan dalam laporan.
- **Bab II: Deskripsi Global** Bab ini memberikan gambaran umum mengenai perangkat lunak yang dibangun, karakteristik pengguna yang terlibat (Admin dan Petugas Gudang), serta batasan-batasan sistem yang berlaku.
- **Bab III: Kebutuhan Spesifik** Bab ini menjabarkan spesifikasi kebutuhan sistem secara rinci, meliputi kebutuhan fungsional, kebutuhan antarmuka eksternal, kebutuhan non-fungsional, serta pemodelan data menggunakan *Entity Relationship Diagram* (ERD).

- **Bab IV: Perancangan Sistem** Bab ini memvisualisasikan alur informasi dan proses dalam sistem melalui *Data Flow Diagram* (DFD) dan *Use Case Diagram*, serta menjelaskan arsitektur sistem yang dirancang.
- **Bab V: Implementasi** Bab ini mendokumentasikan realisasi teknis sistem, mencakup lingkungan pengembangan, struktur kode program, serta penggunaan teknologi utama seperti Next.js dan PostgreSQL dalam pembangunan aplikasi.
- **Bab VI: Pengujian** Bab ini membahas proses validasi perangkat lunak menggunakan metode *Black Box Testing* untuk memastikan bahwa fitur-fitur utama, khususnya fungsi otentikasi, berjalan sesuai dengan spesifikasi kebutuhan.
- **Bab VII: Pemeliharaan** Bab ini menjelaskan strategi perawatan sistem jangka panjang, yang mencakup pemeliharaan korektif, adaptif, preventif, dan penyempurnaan (*perfective maintenance*) guna menjamin keberlanjutan operasional sistem.

BAB II
DESKRIPSI GLOBAL

2.1 Perspektif Produk

Sistem Inventori Barang merupakan produk perangkat lunak yang bersifat mandiri (*standalone*) dan dirancang khusus untuk memfasilitasi pengelolaan persediaan barang secara terkomputerisasi. Sistem ini dikembangkan sebagai solusi independen, bukan merupakan modul atau bagian dari sistem *Enterprise Resource Planning* (ERP) yang lebih besar, namun tetap memiliki fleksibilitas untuk dikembangkan lebih lanjut. Fokus utama produk ini adalah menggantikan mekanisme pencatatan manual dengan sistem digital yang mampu menjamin validitas dan ketersediaan data stok secara *real-time*.

Secara arsitektural, sistem ini mengadopsi model *Client-Server* modern berbasis teknologi web. Sisi server dibangun menggunakan kerangka kerja Next.js yang menangani logika aplikasi dan antarmuka, serta didukung oleh basis data PostgreSQL sebagai media penyimpanan data terpusat yang handal. Implementasi arsitektur ini memberikan keuntungan sentralisasi data, di mana seluruh pemrosesan dan penyimpanan dilakukan di server.

Bagi pengguna akhir, interaksi dengan sistem dilakukan sepenuhnya melalui peramban web (*web browser*) standar tanpa memerlukan instalasi perangkat lunak khusus di sisi klien (*client-side installation*). Model distribusi ini memungkinkan sistem untuk diakses secara fleksibel dari berbagai perangkat komputer yang terhubung dalam jaringan yang sama, selama perangkat tersebut memiliki peramban web yang kompatibel.

2.2 Fungsi Produk

Sistem Inventori Barang menyediakan serangkaian fungsi utama yang dirancang untuk mendukung efisiensi operasional gudang. Secara garis besar, fungsionalitas sistem dikelompokkan ke dalam empat kategori berikut:

- **Otentikasi dan Keamanan** Sistem dilengkapi dengan fitur *Login* yang aman untuk memvalidasi identitas pengguna sebelum memberikan akses masuk. Fitur ini menerapkan mekanisme kontrol akses berbasis peran (*Role-Based Access Control*), yang membedakan hak akses antara Admin dan Petugas Gudang guna menjamin keamanan data.

- **Manajemen Data Master** Fungsi ini memberikan wewenang khusus kepada Admin untuk mengelola data induk yang menjadi referensi sistem. Cakupan fungsi ini meliputi pengelolaan data barang (menambah, mengubah, dan menghapus atribut barang) serta manajemen data pengguna untuk pendaftaran akun staf baru.

- **Transaksi Inventori** Sistem memfasilitasi pencatatan aktivitas pergudangan melalui dua fitur transaksional utama:
    - **Pencatatan Barang Masuk:** Mencatat penerimaan barang yang secara otomatis menambah jumlah stok di basis data.
    - **Pencatatan Barang Keluar:** Mencatat pengeluaran barang yang secara otomatis mengurangi stok, dilengkapi dengan validasi otomatis untuk mencegah pengeluaran barang melebihi stok yang tersedia.

- **Pelaporan (Reporting)** Sistem mampu menyajikan informasi rekapitulasi secara otomatis dalam bentuk laporan. Fungsi ini mencakup penyajian laporan sisa stok terkini (*current stock*) serta riwayat mutasi transaksi (barang masuk dan keluar) dalam periode waktu tertentu untuk keperluan audit dan pemantauan.

2.3 Karakteristik Pengguna

Sistem Inventori Barang dirancang untuk digunakan oleh dua kategori pengguna utama yang memiliki tanggung jawab dan hak akses berbeda dalam operasional sistem. Identifikasi karakteristik pengguna ini bertujuan untuk memastikan antarmuka dan fungsionalitas yang dibangun sesuai dengan kebutuhan masing-masing aktor. Berikut adalah rincian karakteristik pengguna sistem:

1. **Admin**
    - **Peran:** Bertindak sebagai pengelola sistem penuh (*superuser*). Admin memiliki otoritas tertinggi untuk mengakses seluruh fitur sistem, termasuk manajemen data induk (*master data*) barang, pengelolaan akun pengguna, serta akses ke seluruh laporan.
    - **Keahlian:** Pengguna pada kategori ini diharapkan memiliki pemahaman mengenai operasional manajemen data serta kemampuan administrasi pengguna untuk menjaga integritas dan keamanan sistem.

2. **Petugas Gudang (*User*)**
    - **Peran:** Bertindak sebagai operator harian yang berinteraksi langsung dengan fisik barang. Tugas utamanya meliputi input transaksi rutin (barang masuk dan keluar) serta pemantauan ketersediaan stok di lapangan.
    - **Keahlian:** Pengguna ini diharapkan memiliki kemampuan dasar dalam mengoperasikan komputer dan menggunakan peramban web (*web browser*). Selain itu, petugas gudang harus memahami alur logistik atau pergerakan fisik barang di dalam gudang.

Mengingat adanya perbedaan latar belakang teknis di antara pengguna, khususnya pada kategori Petugas Gudang, antarmuka sistem didesain dengan pendekatan yang ramah pengguna (*user-friendly*). Navigasi yang intuitif dan tata letak yang sederhana diterapkan agar sistem dapat dipelajari dan dioperasikan dengan mudah dalam waktu singkat, bahkan oleh pengguna awam yang tidak memiliki latar belakang teknologi informasi mendalam.

2.4 Batasan

Untuk memastikan pengembangan sistem tetap fokus dan dapat diselesaikan tepat waktu, serta untuk menyamakan ekspektasi pengguna terhadap kapabilitas sistem, ditetapkan batasan-batasan sebagai berikut:

1. **Platform dan Aksesibilitas** Sistem Inventori Barang dikembangkan sepenuhnya sebagai aplikasi berbasis web. Oleh karena itu, akses terhadap sistem hanya dapat dilakukan melalui peramban web (*web browser*) modern seperti Google Chrome, Mozilla Firefox, atau Microsoft Edge. Penggunaan sistem ini mutlak membutuhkan koneksi jaringan (intranet atau internet) yang menghubungkan perangkat pengguna dengan server aplikasi.

2. **Lingkup Fungsional** Fokus utama sistem dibatasi pada pengelolaan data inventori internal, khususnya pemantauan kuantitas fisik barang. Sistem ini **tidak** menangani proses bisnis eksternal maupun aspek finansial, yang meliputi:
    - Tidak mencakup fitur pembelian (*purchasing*) ke pemasok (*supplier*).
    - Tidak mencakup fitur penjualan (*sales*) atau kasir (*Point of Sale*) kepada pelanggan.

- Tidak melakukan perhitungan akuntansi, penilaian aset dalam mata uang (Rupiah), maupun perhitungan Laba Rugi.

3. **Integrasi Perangkat Keras** Sistem saat ini belum terintegrasi secara langsung dengan perangkat keras khusus pendukung inventori seperti pemindai kode batang (*barcode scanner*) atau printer label otomatis. Seluruh proses input data, termasuk pencarian barang dan pencatatan transaksi, dilakukan secara manual menggunakan perangkat input standar (papan ketik dan tetikus) untuk menjaga kesederhanaan sistem.

2.5 Asumsi dan Ketergantungan

Agar Sistem Inventori Barang dapat beroperasi sesuai dengan spesifikasi yang diharapkan, pengembangan dan implementasi sistem didasarkan pada sejumlah asumsi dan ketergantungan terhadap faktor eksternal berikut:

1. **Lingkungan Peramban (*Web Browser*)** Diasumsikan bahwa pengguna mengakses sistem menggunakan peramban web modern yang mendukung standar teknologi web terkini, khususnya eksekusi JavaScript. Peramban yang direkomendasikan meliputi Google Chrome, Microsoft Edge, atau Mozilla Firefox. Hal ini merupakan ketergantungan krusial mengingat antarmuka sistem yang dibangun menggunakan kerangka kerja Next.js sangat bergantung pada kinerja JavaScript di sisi klien (*client-side*).

2. **Ketersediaan Server dan Basis Data** Operasional sistem memiliki ketergantungan mutlak pada ketersediaan infrastruktur server. Diasumsikan bahwa layanan *hosting* yang menjalankan aplikasi dan layanan basis data PostgreSQL selalu dalam kondisi aktif dan stabil. Kegagalan pada layanan server atau basis data akan menyebabkan sistem berhenti total dan tidak dapat diakses oleh pengguna.

3. **Konektivitas Jaringan** Mengingat arsitektur sistem yang berbasis web (*client-server*), kinerja dan responsivitas aplikasi sangat bergantung pada stabilitas koneksi jaringan yang menghubungkan perangkat pengguna dengan server. Gangguan pada konektivitas dapat menghambat proses pengiriman data transaksi dan pembaruan informasi stok secara *real-time*.

4. **Kedisiplinan Operasional (SOP)** Validitas data stok dalam sistem sepenuhnya bergantung pada faktor manusia. Diasumsikan bahwa Petugas Gudang mematuhi Standar Operasional Prosedur (SOP) untuk mencatat setiap pergerakan fisik barang (masuk maupun keluar) secara disiplin dan tepat waktu. Sistem tidak memiliki kemampuan untuk melacak pergerakan barang secara fisik yang tidak diinputkan secara manual (*unrecorded*

movement), sehingga kelalaian pencatatan akan berdampak langsung pada ketidakakuratan data laporan.

BAB III
KEBUTUHAN SPESIFIK

3.1 Kebutuhan Antarmuka Eksternal

Bagian ini mendeskripsikan spesifikasi antarmuka yang menghubungkan pengguna dengan perangkat lunak Sistem Inventori Barang. Spesifikasi ini mencakup desain antarmuka pengguna, struktur navigasi, serta mekanisme komunikasi sistem terhadap aksi pengguna untuk memastikan interaksi yang ergonomis dan efisien.

3.1.1 Antarmuka Pengguna (*User Interface*)

Sistem Inventori Barang dirancang sebagai aplikasi berbasis web yang dapat diakses melalui peramban web (*web browser*) standar yang mendukung HTML5 dan CSS3. Desain antarmuka menerapkan prinsip *Responsive Web Design* (RWD), yang memungkinkan tata letak (*layout*) halaman beradaptasi secara otomatis terhadap berbagai resolusi layar, baik pada perangkat desktop, tablet, maupun perangkat seluler. Konsistensi visual dipertahankan melalui penggunaan palet warna yang seragam, tipografi yang mudah dibaca, serta penempatan elemen interaktif yang intuitif guna meminimalkan beban kognitif pengguna saat mengoperasikan sistem.

3.1.2 Struktur Navigasi

Navigasi utama dalam sistem ini diimplementasikan menggunakan pola tata letak *dashboard* yang terdiri dari dua komponen utama: *Sidebar* (bilah samping) dan *Topbar* (bilah atas).

- **Sidebar:** Berfungsi sebagai menu navigasi vertikal yang memuat tautan ke modul-modul utama sistem, seperti manajemen barang, transaksi masuk/keluar, dan laporan. Menu ini bersifat *collapsible* (dapat disembunyikan/dimunculkan) untuk mengoptimalkan ruang kerja pada layar yang lebih kecil.

- **Topbar:** Berfungsi menyajikan informasi kontekstual seperti profil pengguna yang sedang aktif, notifikasi sistem, serta akses cepat ke pengaturan akun.

- **Otentikasi:** Sebelum mengakses navigasi utama, sistem menyediakan antarmuka *Login* sebagai gerbang keamanan. Fitur *Logout* ditempatkan secara strategis pada *Topbar* atau bagian bawah *Sidebar* untuk memudahkan pengguna mengakhiri sesi akses mereka secara aman.

3.1.3 Mekanisme Pesan Sistem (*System Feedback*)

Sistem dilengkapi dengan mekanisme umpan balik interaktif untuk merespons setiap aksi yang dilakukan oleh pengguna. Standarisasi pesan sistem dibagi menjadi dua kategori:

1. **Pesan Kesalahan (*Error Message*):** Sistem akan menampilkan validasi secara *real-time* atau *post-submit* apabila terjadi kesalahan input data (misalnya: format data tidak sesuai atau kolom wajib belum diisi). Pesan kesalahan disampaikan dengan bahasa yang informatif dan spesifik, menjelaskan penyebab kesalahan serta instruksi perbaikan, bukan sekadar kode kesalahan teknis.

2. **Konfirmasi Aksi (*Confirmation Dialog*):** Untuk aksi krusial yang bersifat destruktif atau memengaruhi integritas data (seperti menghapus data barang atau membatalkan transaksi), sistem mewajibkan munculnya jendela konfirmasi (*modal alert*). Hal ini bertujuan untuk mencegah ketidaksengajaan pengguna dalam melakukan perubahan data yang permanen.

3.2 Kebutuhan Fungsional

Kebutuhan fungsional mendefinisikan layanan-layanan spesifik yang harus disediakan oleh sistem, bagaimana sistem bereaksi terhadap input tertentu, dan bagaimana sistem berperilaku dalam situasi tertentu. Identifikasi kebutuhan ini didasarkan pada proses bisnis inventori yang melibatkan dua aktor utama, yaitu **Admin** dan **Petugas Gudang**.

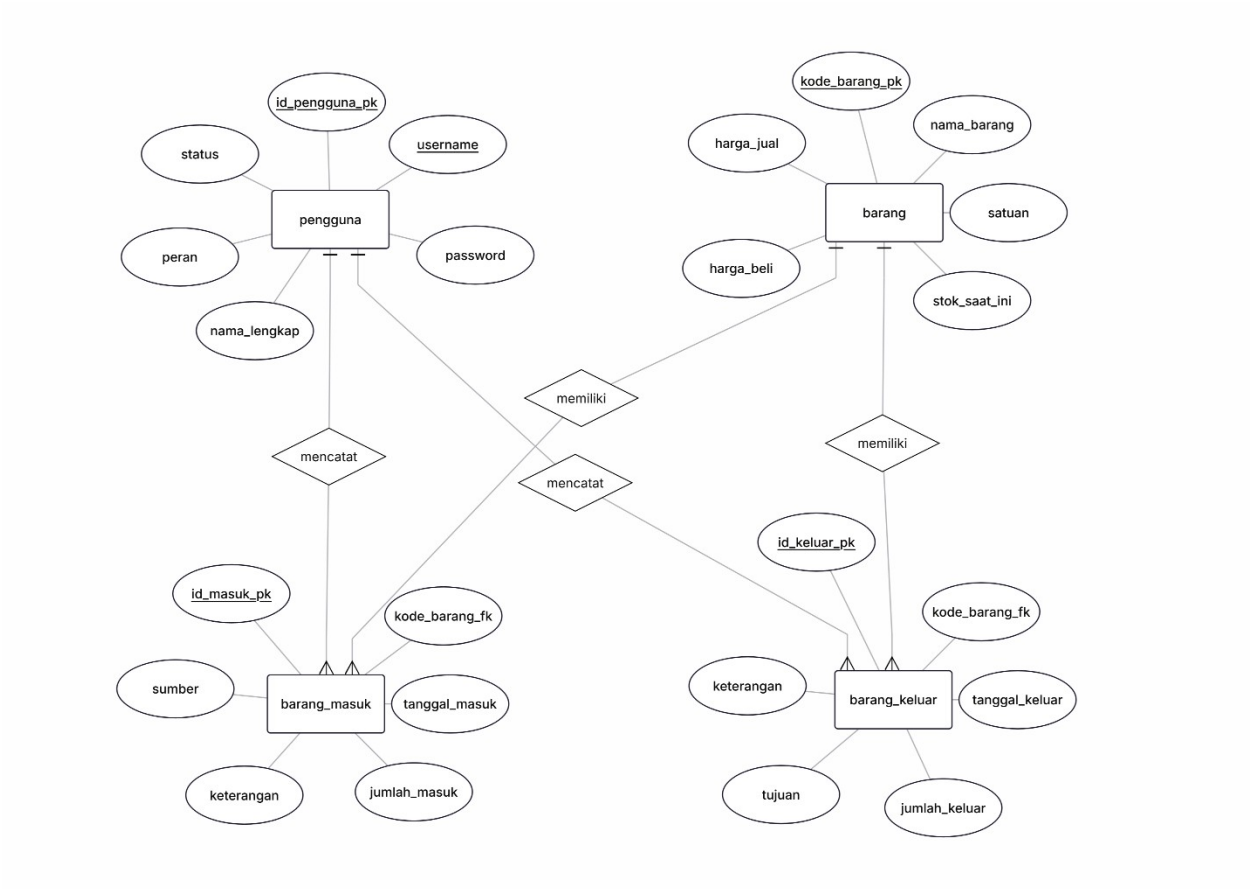
Rincian kebutuhan fungsional Sistem Inventori Barang disajikan dalam Tabel 1 berikut:

Table 1 Rincian kebutuhan fungsional

Kode Kebutuhan	**Nama Fungsi**	**Deskripsi Fungsional**	**Aktor Terkait**
KF-001	Otentikasi Pengguna (*Login*)	Sistem harus memvalidasi identitas pengguna melalui mekanisme input *username* dan *password*. Sistem wajib membatasi hak akses berdasarkan peran (*role*) yang melekat pada akun pengguna setelah login berhasil.	Admin, Petugas Gudang
KF-002	Manajemen Data Pengguna	Sistem menyediakan fasilitas bagi aktor untuk melakukan operasi CRUD (*Create, Read, Update, Delete*) terhadap data pengguna sistem. Hal ini mencakup	Admin

Kode Kebutuhan	Nama Fungsi	Deskripsi Fungsional	Aktor Terkait
		pendaftaran akun baru, pembaruan informasi profil, dan penonaktifan akun.	
KF-003	Manajemen Data Master Barang	Sistem memfasilitasi pengelolaan data induk barang (nama barang, kode unik, kategori, satuan, dan deskripsi). Fungsi ini mencakup penambahan item baru dan pemutakhiran informasi barang, namun tidak mengubah jumlah stok secara langsung (hanya data atribut).	Admin
KF-004	Pencatatan Barang Masuk	Sistem menyediakan antarmuka untuk mencatat transaksi penerimaan barang dari *supplier*. Saat data disimpan, sistem secara otomatis melakukan kalkulasi inkremental (penambahan) terhadap jumlah stok barang terkait di basis data.	Petugas Gudang
KF-005	Pencatatan Barang Keluar	Sistem menyediakan antarmuka untuk mencatat pengeluaran barang. Sistem harus melakukan validasi logika stok sebelum transaksi disimpan: **jika jumlah permintaan melebihi stok tersedia, sistem wajib menolak transaksi** dan menampilkan pesan peringatan. Jika valid, sistem secara otomatis mengurangi stok (*decrement*).	Petugas Gudang
KF-006	Pembuatan Laporan (*Reporting*)	Sistem mampu menampilkandan mencetak laporan rekapitulasi yang mencakup laporan sisa stok terkini (*current stock*) dan riwayat mutasi barang (transaksi masuk dan keluar) dalam periode waktu tertentu.	Admin

3.3 Deskripsi Data



Gambar 1 ERD

Pemodelan data dalam pengembangan Sistem Inventori Barang dirancang untuk merepresentasikan struktur informasi yang akan dikelola oleh basis data. Pendekatan perancangan data menggunakan *Entity Relationship Diagram* (ERD) untuk memvisualisasikan entitas-entitas utama beserta hubungan antar entitas tersebut. Struktur data ini menjamin efisiensi penyimpanan dan kemudahan dalam pengambilan informasi.

Berikut adalah deskripsi entitas-entitas utama yang membentuk arsitektur data sistem:

1. **Entitas Pengguna (*Users*)** Entitas ini berfungsi sebagai penyimpanan data kredensial dan profil aktor yang memiliki hak akses ke dalam sistem. Atribut yang dikelola meliputi identitas unik pengguna, *username*, kata sandi (*password*) yang terenkripsi, serta level otorisasi (*role*) apakah sebagai Admin atau Petugas Gudang. Data pada entitas ini menjadi acuan utama dalam proses validasi saat *login*.

2. **Entitas Barang (*Items/Inventory*)** Entitas ini merepresentasikan data induk (*master data*) dari seluruh inventori yang dikelola. Fungsinya adalah menyimpan atribut statis dan

dinamis dari suatu barang, seperti kode barang, nama barang, kategori, satuan unit, dan jumlah stok terkini. Entitas ini menjadi pusat referensi bagi entitas transaksional lainnya.

3. **Entitas Transaksi (*Transactions*)** Untuk mengakomodasi dinamika stok, sistem memisahkan pencatatan ke dalam dua entitas logis, yaitu **Barang Masuk** dan **Barang Keluar**.
    - **Barang Masuk:** Menyimpan riwayat penambahan stok, mencakup informasi tanggal penerimaan, pemasok (*supplier*), dan jumlah barang yang diterima.
    - **Barang Keluar:** Menyimpan riwayat pengurangan stok, mencakup informasi tanggal keluar, tujuan pengiriman/penggunaan, dan jumlah barang yang dikeluarkan.

Integritas Data dan Relasi Untuk menjamin konsistensi dan validitas informasi, setiap entitas dilengkapi dengan *Primary Key* (Kunci Utama) yang berfungsi sebagai pengidentifikasi unik untuk setiap rekaman data (*record*), sehingga mencegah duplikasi data. Hubungan antar entitas dibangun menggunakan mekanisme *Foreign Key* (Kunci Asing) untuk menjaga integritas referensial. Sebagai contoh, entitas Transaksi Barang Masuk dan Barang Keluar harus terhubung dengan *Primary Key* dari entitas Barang. Hal ini memastikan bahwa sistem tidak akan mencatat transaksi untuk barang yang tidak terdaftar dalam data induk, serta meminimalkan anomali data saat dilakukan operasi manipulasi data.

3.4 Kebutuhan Non-Fungsional

Kebutuhan non-fungsional mendefinisikan batasan layanan dan kendala operasional yang harus dipenuhi oleh sistem. Kebutuhan ini mencakup aspek kualitas perangkat lunak seperti kinerja, keamanan, dan keandalan sistem guna memastikan sistem dapat beroperasi dengan standar yang dapat diterima oleh lingkungan bisnis pengguna.

Rincian kebutuhan non-fungsional Sistem Inventori Barang didefinisikan dalam Tabel 3.2 berikut:

Table 2 Rincian kebutuhan non-fungsional

Kode Parameter	**Kategori**	**Deskripsi Kebutuhan**
NFR-001	Kinerja (*Performance*)	Sistem harus memiliki latensi yang rendah dalam pemrosesan data. Pembaruan jumlah stok di basis data dan tampilan antarmuka harus terjadi secara *real-time* (seketika) segera setelah transaksi barang masuk atau keluar berhasil disimpan, untuk menjamin keakuratan

Kode Parameter	Kategori	Deskripsi Kebutuhan
		data bagi pengguna lain yang sedang mengakses sistem secara bersamaan.
NFR-002	Keamanan (*Security*)	Sistem wajib menerapkan mekanisme keamanan berlapis yang mencakup: 1. **Otentikasi:** Verifikasi identitas pengguna melalui *username* dan *password* (disimpan dalam bentuk *hash* terenkripsi). 2. **Otorisasi:** Penerapan hak akses berbasis peran (*Role-Based Access Control*) untuk membatasi fitur yang dapat diakses oleh Admin dan Petugas Gudang secara tegas.
NFR-003	Keandalan (*Reliability*)	Sistem harus memiliki tingkat ketersediaan (*availability*) yang tinggi dan beroperasi secara stabil tanpa kegagalan kritis (*crash*) selama jam operasional perusahaan. Sistem juga harus memiliki mekanisme penanganan kesalahan (*error handling*) yang mencegah sistem berhenti total saat menerima input data yang tidak valid.

3.5 Atribut Kualitas (ISO/IEC 25010)

Untuk menjamin mutu perangkat lunak yang dihasilkan, evaluasi sistem mengacu pada standar kualitas perangkat lunak ISO/IEC 25010. Model kualitas ini mengategorikan atribut produk perangkat lunak menjadi karakteristik-karakteristik utama. Dalam pengembangan Sistem Inventori Barang ini, fokus utama kualitas ditekankan pada tiga karakteristik berikut:

1. **Kesesuaian Fungsional (*Functional Suitability*)** Karakteristik ini mengukur sejauh mana fungsi-fungsi sistem memenuhi kebutuhan yang dinyatakan dan tersirat saat digunakan dalam kondisi tertentu.
    - **Kelengkapan Fungsional:** Sistem telah mengimplementasikan seluruh himpunan fitur yang didefinisikan pada analisis kebutuhan (Sub-bab 3.2), mencakup manajemen data induk, transaksi barang masuk/keluar, dan pelaporan.

- **Ketepatan Fungsional:** Sistem mampu memberikan hasil yang presisi, seperti kalkulasi stok otomatis yang akurat setelah terjadi transaksi, tanpa adanya *bug* logika yang menyebabkan selisih data.

2. **Kebergunaan (*Usability*)** Karakteristik ini berkaitan dengan kemudahan pengguna dalam mengoperasikan sistem demi mencapai tujuan tertentu dengan efektif dan efisien.
    - **Dapat Dipelajari (*Learnability*):** Antarmuka pengguna dirancang dengan tata letak yang sederhana dan navigasi yang intuitif, sehingga pengguna awam (seperti petugas gudang yang mungkin tidak memiliki latar belakang IT mendalam) dapat memahami cara penggunaan sistem dalam waktu singkat.
    - **Perlindungan Kesalahan Pengguna:** Sistem menyediakan pesan kesalahan yang jelas dan mekanisme konfirmasi (seperti dialog konfirmasi sebelum hapus) untuk mencegah kesalahan operasional yang tidak disengaja.

3. **Keamanan (*Security*)** Karakteristik ini menjamin perlindungan informasi dan data sehingga orang atau sistem lain hanya dapat mengakses data sesuai dengan tingkat otorisasi mereka.
    - **Kerahasiaan (*Confidentiality*):** Data sensitif, khususnya kata sandi (*password*) pengguna, tidak disimpan dalam bentuk teks biasa (*plain text*), melainkan melalui proses *hashing* menggunakan algoritma kriptografi yang aman. Hal ini mencegah kebocoran kredensial jika basis data diakses oleh pihak yang tidak berwenang.
    - **Akuntabilitas:** Setiap aktivitas akses sistem diawali dengan proses *login* yang memisahkan hak akses antara Admin dan Petugas Gudang, memastikan bahwa fitur-fitur kritis (seperti manipulasi data master) hanya dapat diakses oleh entitas yang berhak.

3.6 Tools dan Teknologi yang Digunakan

Implementasi Sistem Inventori Barang dibangun dengan memanfaatkan serangkaian perangkat lunak dan teknologi modern. Pemilihan arsitektur teknologi ini didasarkan pada pertimbangan performa, skalabilitas, keamanan, serta dukungan komunitas pengembang yang luas untuk memastikan keberlanjutan sistem (*maintainability*).

Berikut adalah rincian spesifikasi teknologi yang dikelompokkan berdasarkan fungsinya:

3.6.1 Bahasa Pemrograman & Framework

- **Next.js (Fullstack Framework):** Sistem ini dikembangkan menggunakan Next.js, sebuah kerangka kerja berbasis React yang memungkinkan pengembangan aplikasi web secara *fullstack*. Next.js menangani sisi antarmuka pengguna (*frontend*) yang interaktif sekaligus

logika bisnis sisi server (*backend*) melalui fitur *API Routes* dan *Server Actions*. Penggunaan teknologi ini memberikan keuntungan berupa performa tinggi melalui mekanisme *Server-Side Rendering* (SSR) dan pengelolaan kode yang lebih terpadu dalam satu repositori.

3.6.2 Basis Data (*Database*)

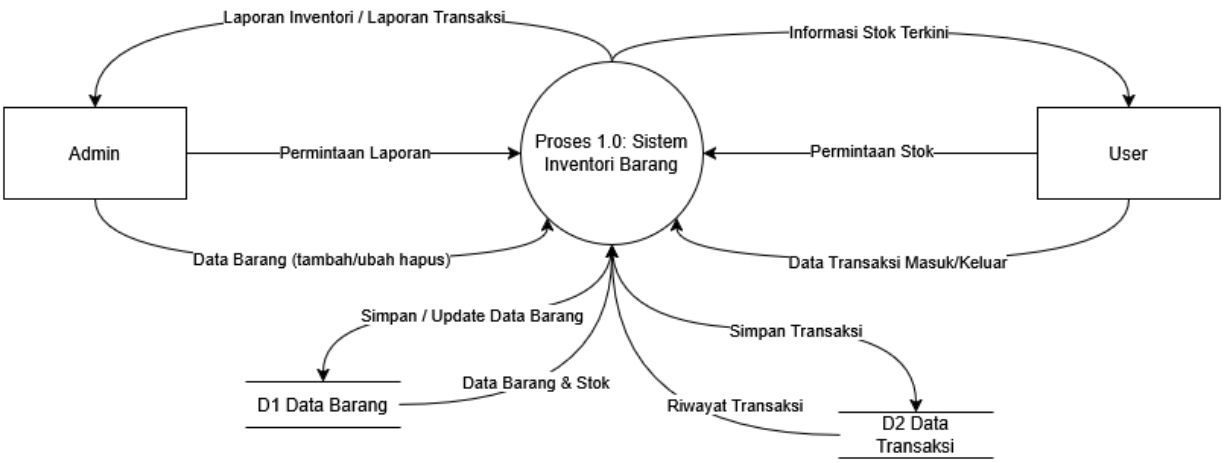
- **PostgreSQL:** PostgreSQL dipilih sebagai Sistem Manajemen Basis Data Relasional (RDBMS) untuk penyimpanan data persisten. Teknologi ini merupakan perangkat lunak *open-source* yang dikenal memiliki stabilitas tinggi, kepatuhan ketat terhadap standar ACID (*Atomicity*, *Consistency*, *Isolation*, *Durability*), serta kemampuan ekstensif dalam menangani relasi data yang kompleks dan integritas data transaksional yang dibutuhkan oleh sistem inventori.

3.6.3 Perangkat Lunak Pengembangan (*Development Tools*)

- **Visual Studio Code (VS Code):** Digunakan sebagai editor kode utama atau *Integrated Development Environment* (IDE). VS Code dipilih karena ekosistem ekstensinya yang kaya, fitur *debugging* yang terintegrasi, serta dukungannya yang optimal terhadap sintaks JavaScript dan TypeScript dalam ekosistem Next.js.
- **Git:** Diterapkan sebagai sistem kontrol versi (*Version Control System*) terdistribusi. Git berfungsi untuk melacak riwayat perubahan kode sumber, memfasilitasi manajemen percabangan (*branching*) fitur, serta memitigasi risiko kehilangan kode selama proses pengembangan berlangsung.

BAB IV
DESKRIPSI PROSES DAN ALIRAN DATA

4.1 Diagram Konteks (Level 0)



Gambar 2 DFD Level 0

Diagram konteks atau DFD Level 0 merepresentasikan pandangan umum terhadap batasan sistem (*scope*). Diagram ini menggambarkan hubungan antara proses utama, yaitu **Proses 1.0: Sistem Inventori Barang**, dengan entitas eksternal dan media penyimpanan data yang terlibat.

Berdasarkan perancangan, terdapat dua entitas eksternal yang berinteraksi langsung dengan sistem, yaitu **Admin** dan **User**. Aliran data yang terjadi antara entitas, proses, dan penyimpanan data dijelaskan sebagai berikut:

4.1.1 Interaksi Entitas Admin

Admin bertindak sebagai pengelola utama data induk dan pelaporan. Interaksi data yang terjadi meliputi:

- **Aliran Masuk (Input):** Admin memberikan masukan berupa **'Data Barang (tambah/ubah hapus)'** untuk mengelola data master inventori. Selain itu, Admin juga mengirimkan **'Permintaan Laporan'** guna memicu sistem memproses rekapitulasi data.
- **Aliran Keluar (Output):** Sebagai respons atas permintaan tersebut, sistem menyajikan **'Laporan Inventori / Laporan Transaksi'** yang berisi informasi status barang dan riwayat mutasi kepada Admin.

4.1.2 Interaksi Entitas User

Entitas User merepresentasikan aktor operasional yang menangani aktivitas harian gudang. Aliran data pada entitas ini meliputi:

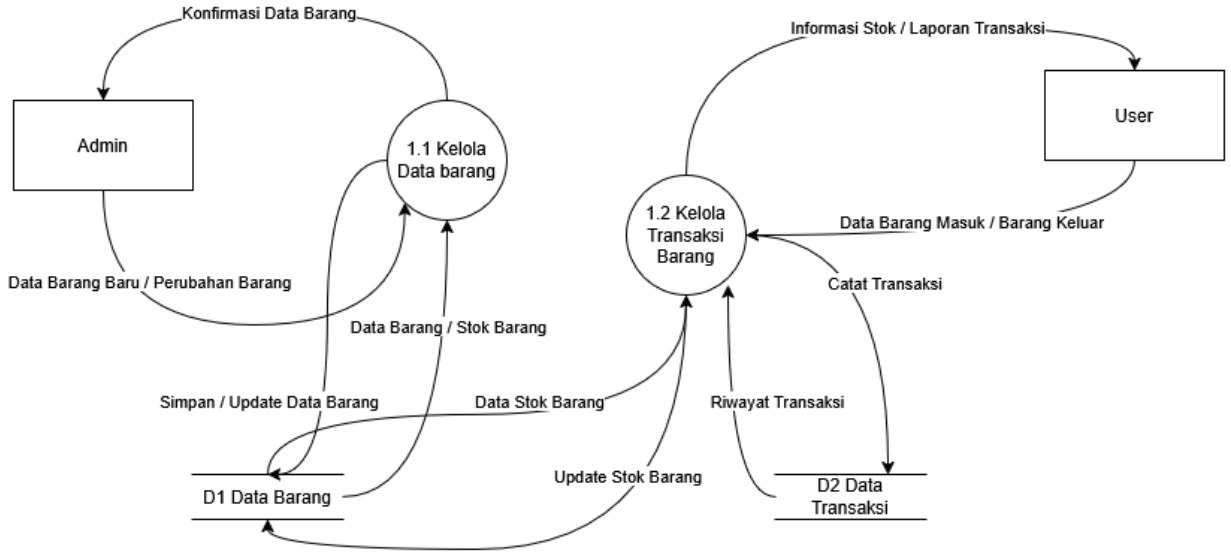
- **Aliran Masuk (Input):** User memasukkan **'Data Transaksi Masuk/Keluar'** untuk mencatat pergerakan barang secara *real-time*. User juga dapat mengirimkan **'Permintaan Stok'** untuk mengetahui ketersediaan barang tertentu.
- **Aliran Keluar (Output):** Sistem merespons permintaan user dengan menampilkan **'Informasi Stok Terkini'** untuk memastikan akurasi data sebelum transaksi dilakukan.

4.1.3 Interaksi dengan Penyimpanan Data (*Data Store*)

Proses 1.0 terintegrasi dengan dua penyimpanan data utama untuk menjaga persistensi data:

1. **D1 Data Barang:** Sistem melakukan proses **'Simpan / Update Data Barang'** ke dalam penyimpanan ini setiap kali ada pembaruan dari Admin atau perubahan stok otomatis. Sebaliknya, sistem mengambil **'Data Barang & Stok'** dari D1 untuk kebutuhan validasi dan pelaporan.
2. **D2 Data Transaksi:** Setiap aktivitas mutasi barang yang dilakukan oleh User akan direkam melalui aliran **'Simpan Transaksi'** ke dalam D2. Saat pelaporan dibutuhkan, sistem akan mengambil data kembali berupa **'Riwayat Transaksi'** dari penyimpanan ini.

4.2 *Data Flow Diagram* (DFD) Level 1



Gambar 3 DFD Level 1

Diagram Level 1 merupakan dekomposisi lebih lanjut dari *Proses 1.0* pada Diagram Konteks. Diagram ini menjabarkan rincian pemrosesan data menjadi dua sub-proses utama, yaitu

pengelolaan data induk barang dan pengelolaan transaksi mutasi barang. Pemisahan ini bertujuan untuk memperjelas batas tanggung jawab antara manipulasi data statis dan data transaksional.

Berdasarkan diagram alur data Level 1, deskripsi rinci untuk setiap proses adalah sebagai berikut:

4.2.1 Proses 1.1: Kelola Data Barang

Proses ini didedikasikan untuk manajemen data induk (*master data*) yang menjadi referensi utama sistem. Aktor yang memiliki otoritas penuh terhadap proses ini adalah **Admin**.

- **Aliran Masuk:** Interaksi dimulai ketika Admin mengirimkan aliran data **'Data Barang Baru / Perubahan Barang'**. Aliran ini berisi atribut detail barang yang akan didaftarkan atau diperbarui.
- **Interaksi Penyimpanan Data (D1):** Proses 1.1 kemudian meneruskan data tersebut ke penyimpanan *D1 Data Barang* melalui aliran **'Simpan / Update Data Barang'** untuk menjamin persistensi data. Sebaliknya, proses juga mengambil **'Data Barang / Stok Barang'** dari D1 untuk keperluan verifikasi atau penamilan data saat disunting.
- **Aliran Keluar:** Setelah operasi basis data berhasil, sistem memberikan umpan balik kepada Admin berupa **'Konfirmasi Data Barang'** yang menandakan bahwa data telah tersimpan dengan aman.

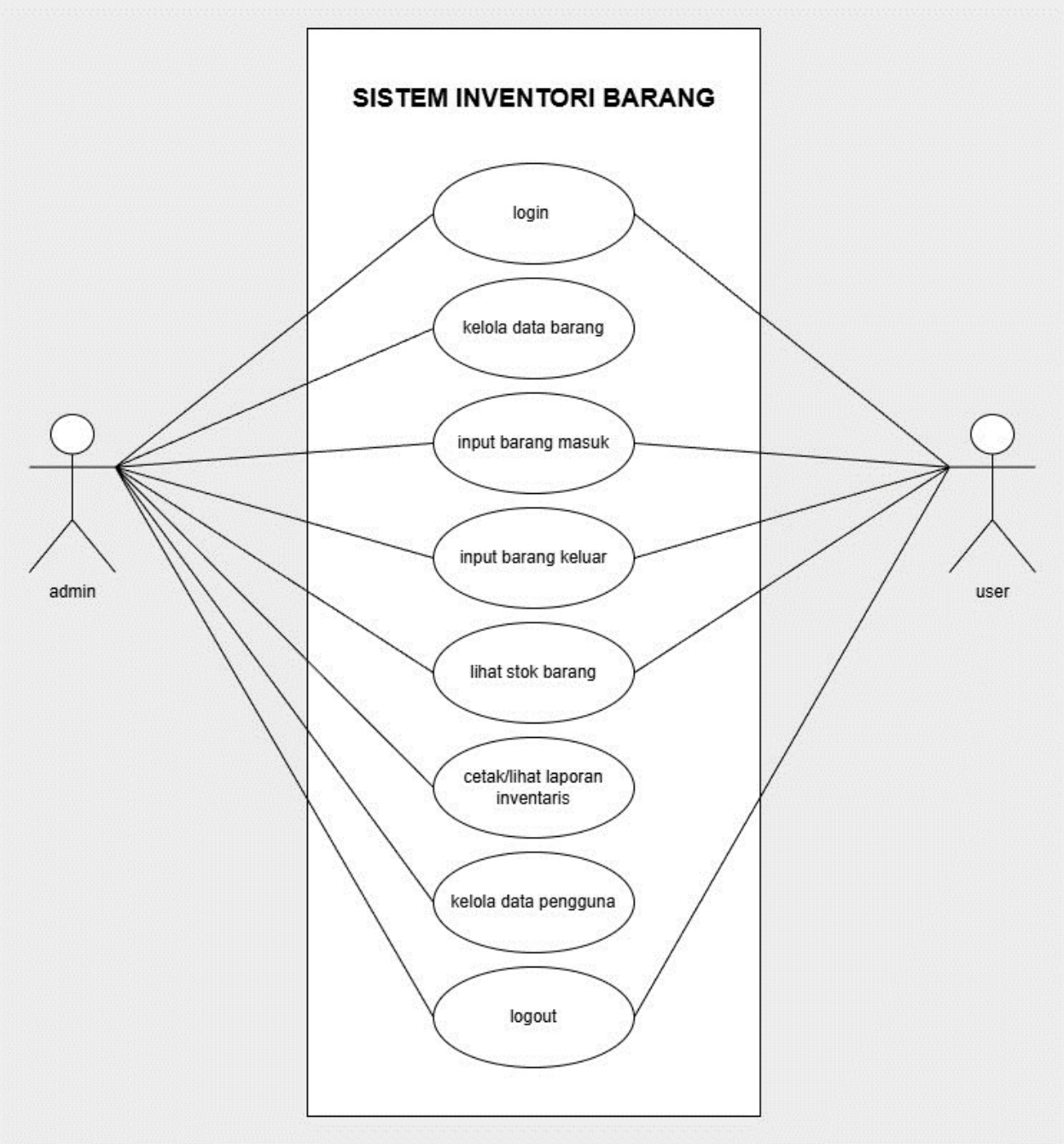
4.2.2 Proses 1.2: Kelola Transaksi Barang

Proses ini menangani aktivitas operasional harian yang melibatkan mutasi stok. Aktor utama yang berinteraksi dalam proses ini adalah **User** (Petugas Gudang).

- **Aliran Masuk:** User menginisiasi proses dengan memasukkan **'Data Barang Masuk / Barang Keluar'**. Input ini mencakup informasi mengenai jumlah barang, jenis transaksi, dan tanggal kejadian.
- **Validasi dan Interaksi Stok (D1):** Sebelum transaksi diproses, sistem mengambil **'Data Stok Barang'** dari *D1 Data Barang* untuk memvalidasi ketersediaan stok (khususnya untuk transaksi barang keluar).
- **Mekanisme Pembaruan Otomatis (D1):** Fitur krusial pada proses ini adalah kemampuan sistem melakukan kalkulasi otomatis. Setelah transaksi valid, proses mengirimkan aliran **'Update Stok Barang'** kembali ke *D1 Data Barang*. Hal ini memastikan jumlah stok pada data induk selalu *up-to-date* sesuai dengan realitas fisik di gudang.
- **Pencatatan Riwayat (D2):** Secara simultan, proses melakukan perekaman data historis melalui aliran **'Catat Transaksi'** ke penyimpanan *D2 Data Transaksi*. Data ini dapat dipanggil kembali melalui aliran **'Riwayat Transaksi'** untuk kebutuhan audit atau pelaporan.

- **Aliran Keluar:** Sebagai hasil akhir proses, sistem menyajikan **'Informasi Stok / Laporan Transaksi'** kepada User sebagai bukti bahwa mutasi barang telah berhasil dieksekusi dan dicatat.

4.3 *Use Case Diagram* dan Deskripsi



Gambar 4 Usecase

Use Case Diagram digunakan untuk memodelkan fungsionalitas sistem yang diharapkan dan bagaimana pengguna berinteraksi dengan fungsi-fungsi tersebut. Diagram ini menggambarkan batasan sistem serta hubungan antara aktor (pengguna) dengan *use case* (fitur sistem). Visualisasi interaksi antara aktor dan fungsionalitas dalam Sistem Inventori Barang dapat dilihat pada Gambar 4.3.

4.3.1 Identifikasi Aktor

Berdasarkan analisis kebutuhan sistem, terdapat dua aktor utama yang berinteraksi dengan sistem. Definisi peran masing-masing aktor adalah sebagai berikut:

1. **Admin:** Aktor ini merepresentasikan pengguna dengan tingkat otoritas tertinggi (*superuser*). Admin memiliki hak akses penuh terhadap seluruh fungsionalitas sistem, termasuk manajemen data induk dan manajemen pengguna lain.
2. **User (Petugas Gudang):** Aktor ini merepresentasikan pengguna operasional dengan hak akses terbatas. Peran User difokuskan pada kegiatan operasional harian gudang seperti pencatatan transaksi dan pemantauan stok, tanpa izin untuk mengubah konfigurasi sistem atau data induk krusial.

4.3.2 Definisi Use Case

Berikut adalah deskripsi fungsional untuk setiap *use case* yang terdapat dalam diagram:

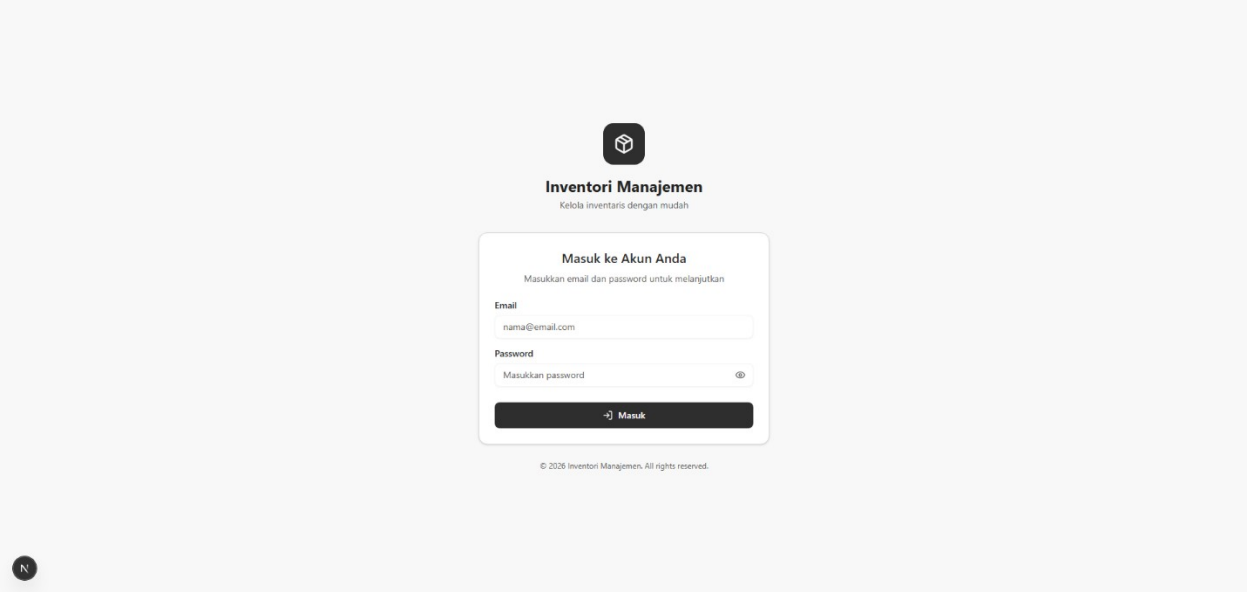
1. **Login:** Proses otentikasi awal yang mewajibkan aktor memasukkan kredensial (*username* dan *password*) untuk memvalidasi identitas sebelum mendapatkan akses masuk ke dalam sistem.
2. **Kelola Data Barang:** Fungsionalitas administratif yang memungkinkan aktor untuk melakukan operasi manajemen data induk barang, meliputi penambahan barang baru, perubahan informasi barang, atau penghapusan data barang yang sudah tidak relevan. **(Akses khusus Admin)**.
3. **Input Barang Masuk:** Fitur operasional untuk mencatat transaksi penerimaan barang. Sistem akan menambah jumlah stok secara otomatis berdasarkan input ini.
4. **Input Barang Keluar:** Fitur operasional untuk mencatat transaksi pengeluaran barang. Sistem akan mengurangi jumlah stok dan melakukan validasi ketersediaan barang sebelum transaksi disimpan.
5. **Lihat Stok Barang:** Fitur pemantauan yang memungkinkan aktor melihat informasi jumlah persediaan barang terkini (*real-time*) yang tersedia di gudang.
6. **Cetak/Lihat Laporan Inventaris:** Fungsionalitas untuk menghasilkan rekapitulasi data dalam bentuk laporan, baik laporan sisa stok maupun riwayat mutasi barang, yang dapat dilihat di layar atau dicetak untuk kebutuhan administrasi.

7. **Kelola Data Pengguna:** Fungsionalitas administratif untuk manajemen akun pengguna sistem, termasuk pembuatan akun baru untuk petugas gudang, pengaturan ulang kata sandi, atau penonaktifan akun. **(Akses khusus Admin)**.
8. **Logout:** Proses pengakhiran sesi akses pengguna secara aman untuk mencegah penyalahgunaan akun oleh pihak yang tidak berwenang setelah penggunaan sistem selesai.

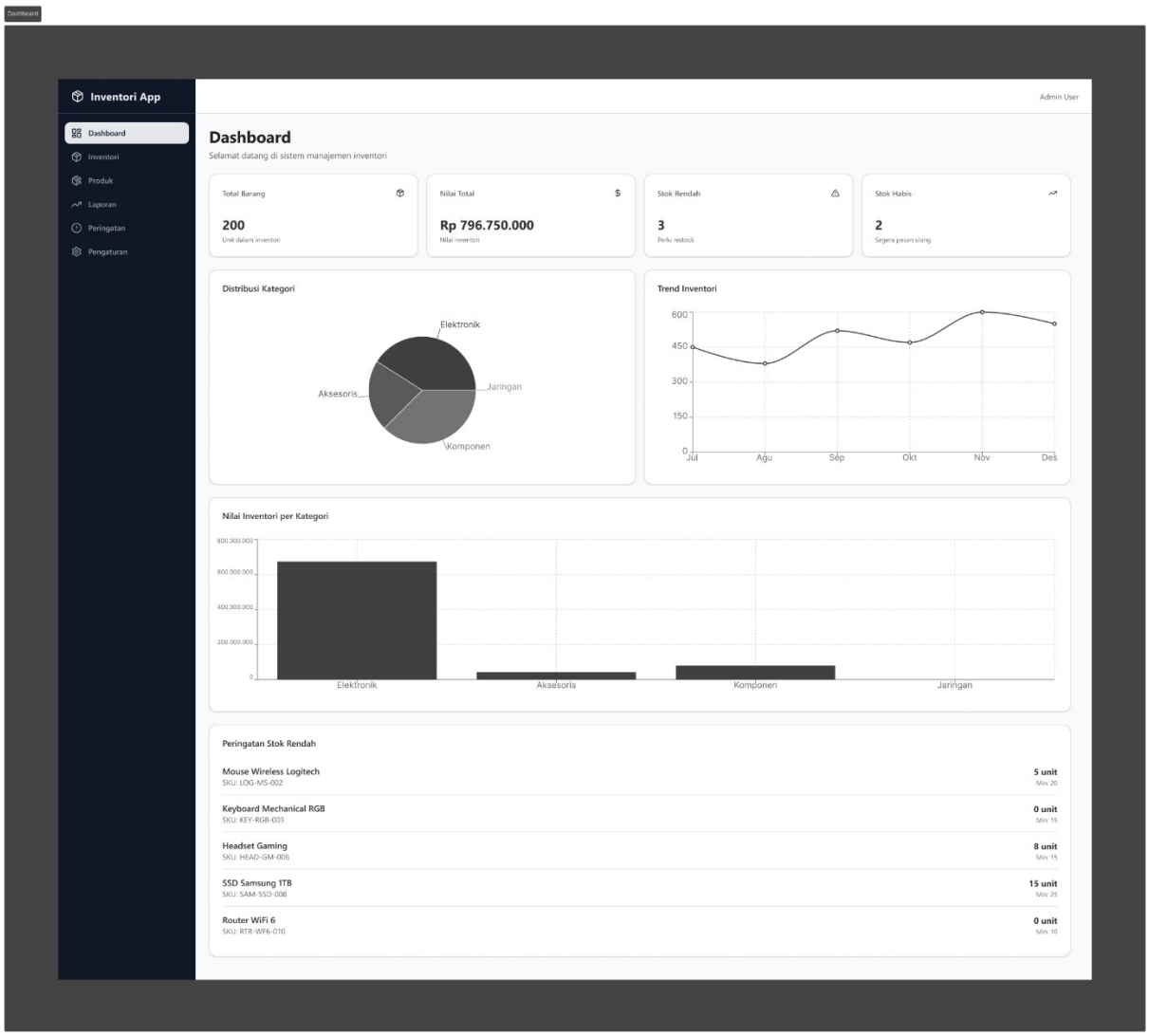
4.4 Deskripsi Teknis Sistem

- Menjelaskan arsitektur yang digunakan, yaitu arsitektur *Client-Server* (Web-based).
- Menjelaskan komponen-komponen yang terlibat (Browser, Web Server, Application Server, Database Server).
- Deskripsi singkat mengenai teknologi *Front-end* (HTML, CSS, JavaScript) dan *Back-end* (misalnya Python/Flask).

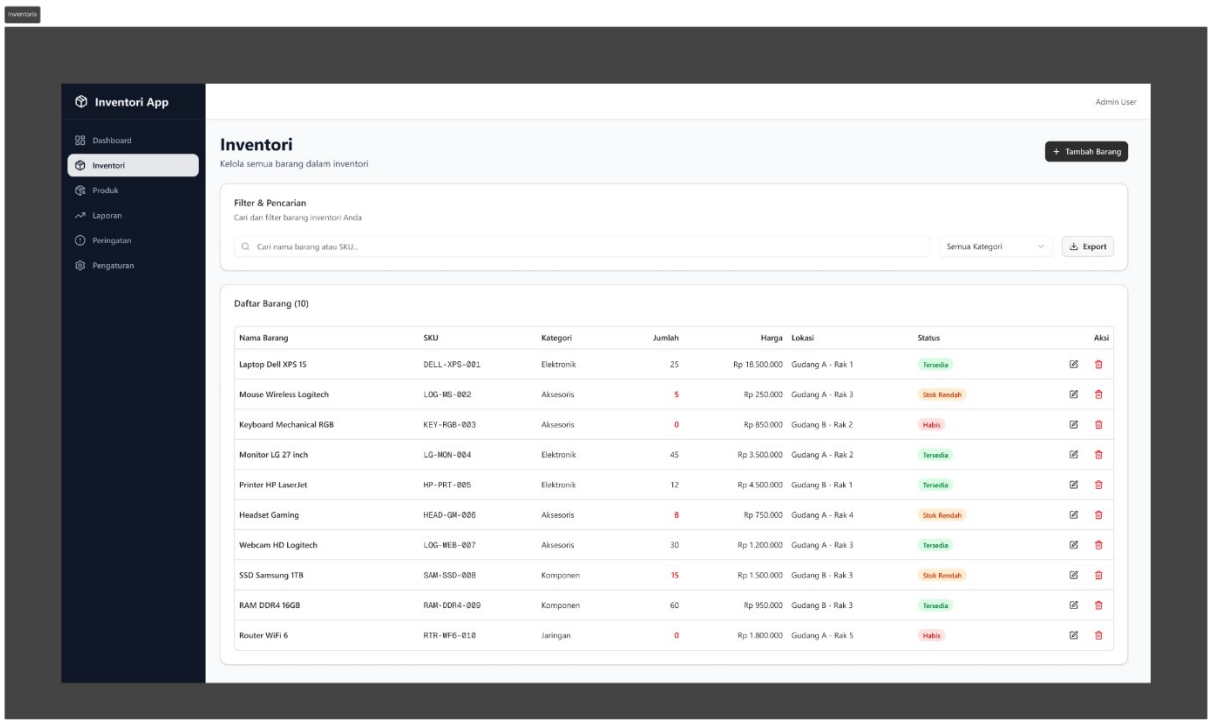
4.5 Perancangan Antarmuka



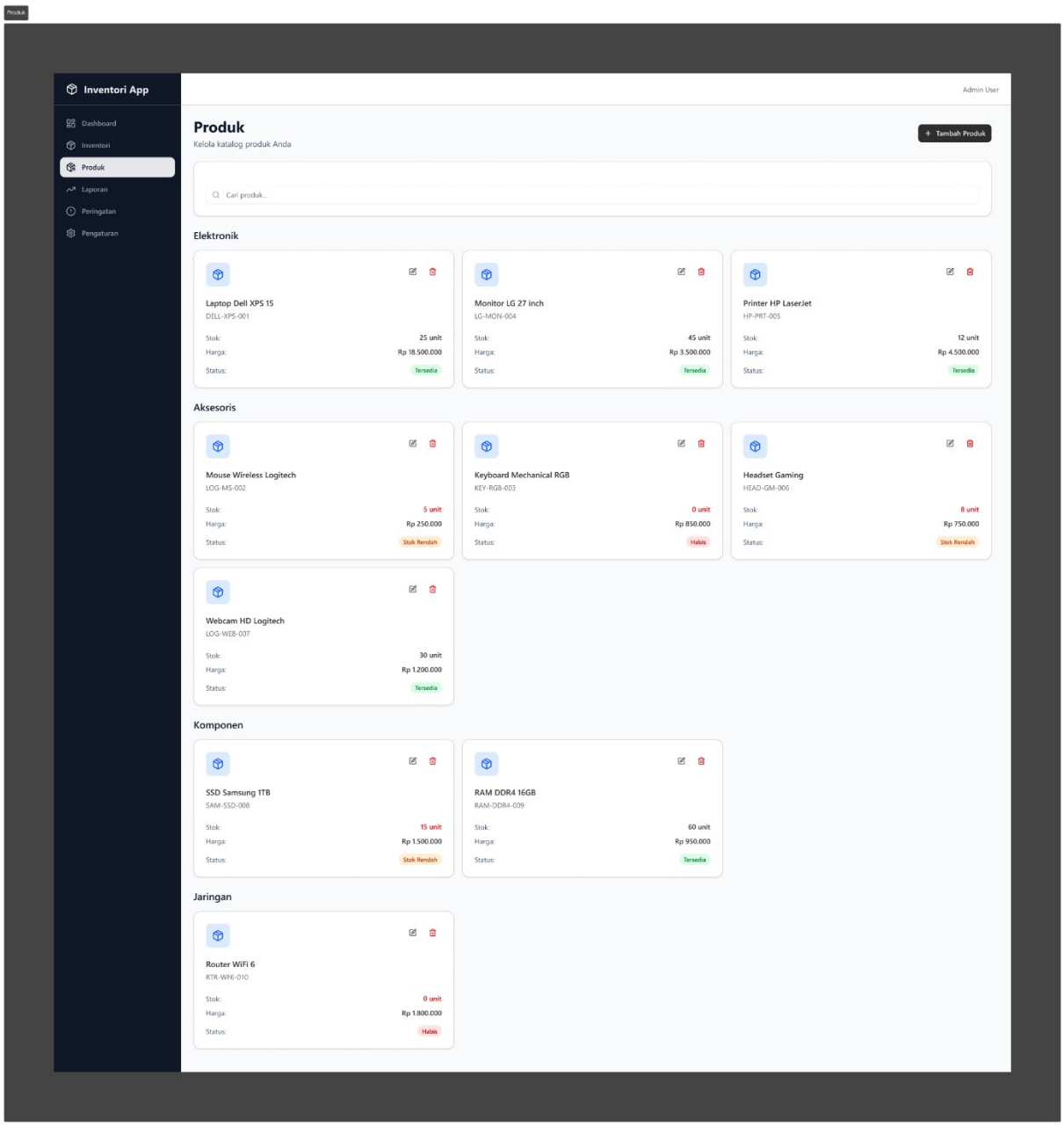
Gambar 5 Desain halaman login



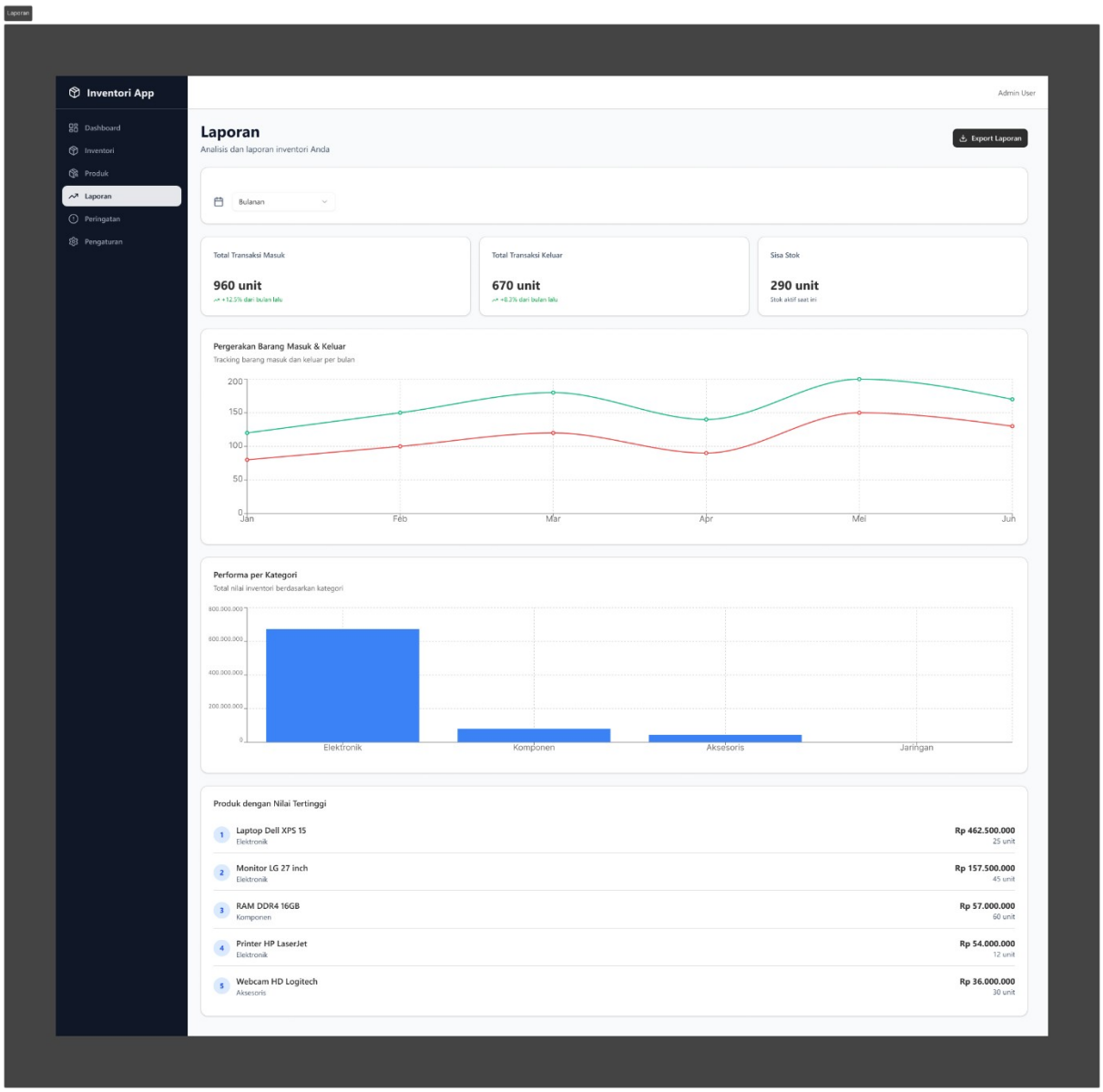
Gambar 6 Desain halaman dashboard



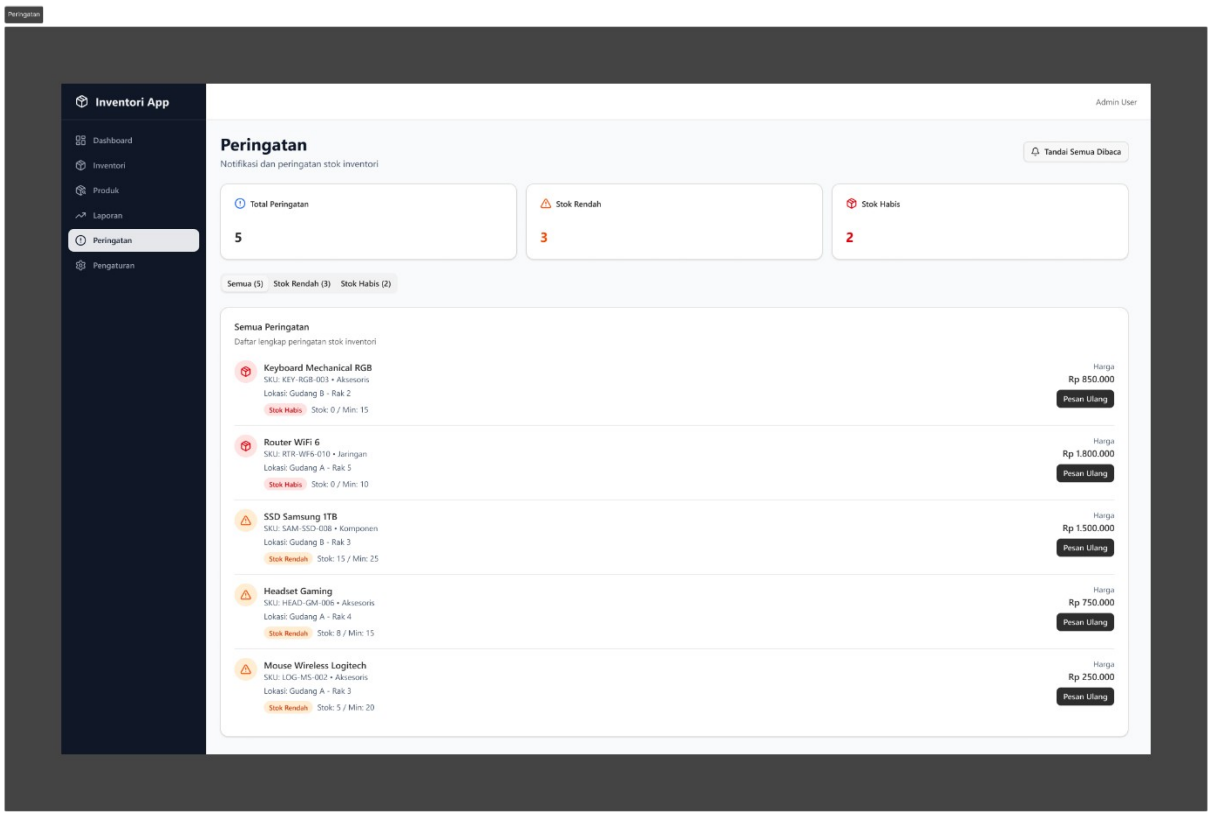
Gambar 7 Desain dalaman inventori



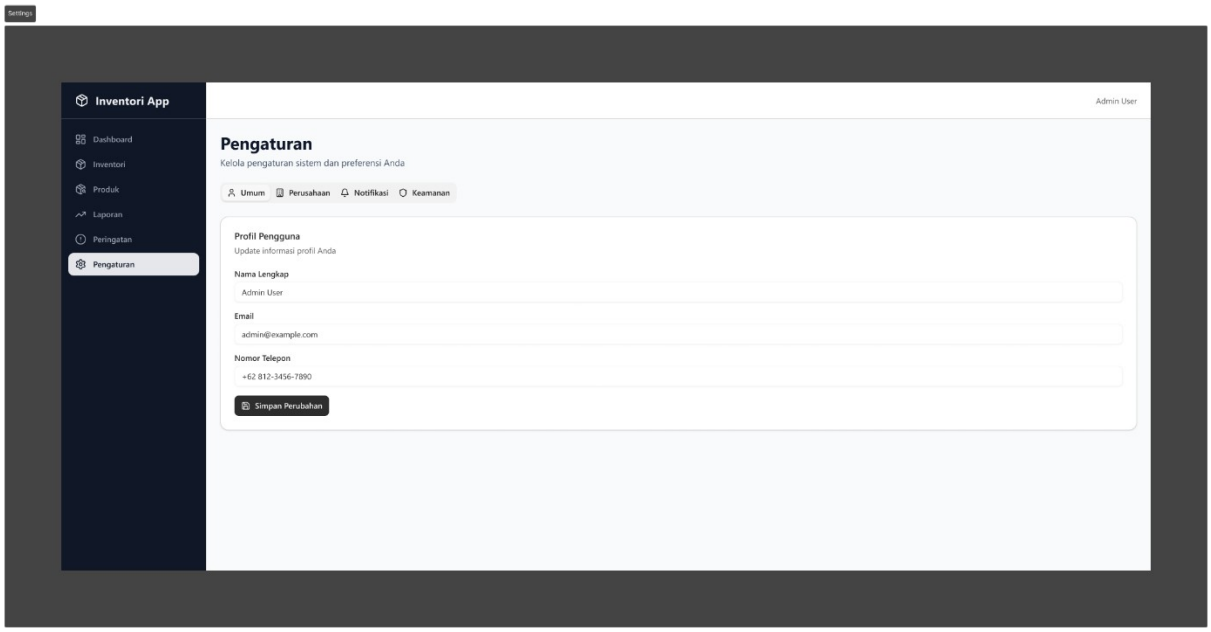
Gambar 8 Desain halaman produk



Gambar 9 Desain halaman laporan



Gambar 10 Desain halaman peringatan



Gambar 11 Desain halaman pengaturan

BAB V
IMPLEMENTASI SISTEM

5.1 Tujuan Implementasi

Tahap implementasi merupakan fase krusial dalam siklus hidup pengembangan perangkat lunak (*Software Development Life Cycle*), di mana seluruh spesifikasi kebutuhan (Bab III) dan rancangan arsitektur sistem (Bab IV) ditransformasikan menjadi kode program (*source code*) yang dapat dieksekusi. Tujuan utama dari bab ini adalah mendokumentasikan proses realisasi teknis pembangunan 'Sistem Inventori Barang' hingga menjadi perangkat lunak yang fungsional.
Secara spesifik, proses implementasi ini mencakup serangkaian tahapan teknis yang sistematis, antara lain:

1. **Konfigurasi Lingkungan (*Environment Setup*):** Penyiapan infrastruktur pengembangan, termasuk instalasi *runtime*, inisialisasi proyek Next.js, serta konfigurasi koneksi basis data PostgreSQL.
2. **Konstruksi Basis Data:** Penerjemahan desain ERD menjadi skema fisik basis data melalui proses migrasi, serta penyiapan data awal (*seeding*).
3. **Pengkodean Logika Bisnis (*Business Logic Implementation*):** Penulisan kode program sisi server (*backend*) menggunakan bahasa TypeScript untuk menangani aturan bisnis, validasi input, dan manipulasi data.
4. **Pembangunan Antarmuka (*User Interface Construction*):** Pengembangan komponen visual yang responsif dan interaktif pada sisi klien (*frontend*) sesuai dengan desain antarmuka yang telah dirancang.
5. **Integrasi Sistem:** Penggabungan komponen *frontend*, *backend*, dan basis data untuk memastikan aliran data berjalan sesuai dengan alur proses yang diharapkan.

Hasil akhir dari tahap implementasi ini adalah sebuah rilis kandidat aplikasi berbasis web yang telah memenuhi standar fungsionalitas dan siap untuk memasuki fase pengujian sistem (*testing phase*) guna memverifikasi keandalan dan kinerjanya sebelum diserahkan kepada pengguna akhir.

5.2 Lingkungan Pengembangan

Lingkungan pengembangan (*development environment*) mencakup spesifikasi perangkat keras dan perangkat lunak yang digunakan untuk menulis, menguji, dan men-deploy kode program. Standarisasi lingkungan ini penting untuk memastikan kompatibilitas sistem dan mencegah terjadinya *error* yang disebabkan oleh perbedaan konfigurasi sistem.

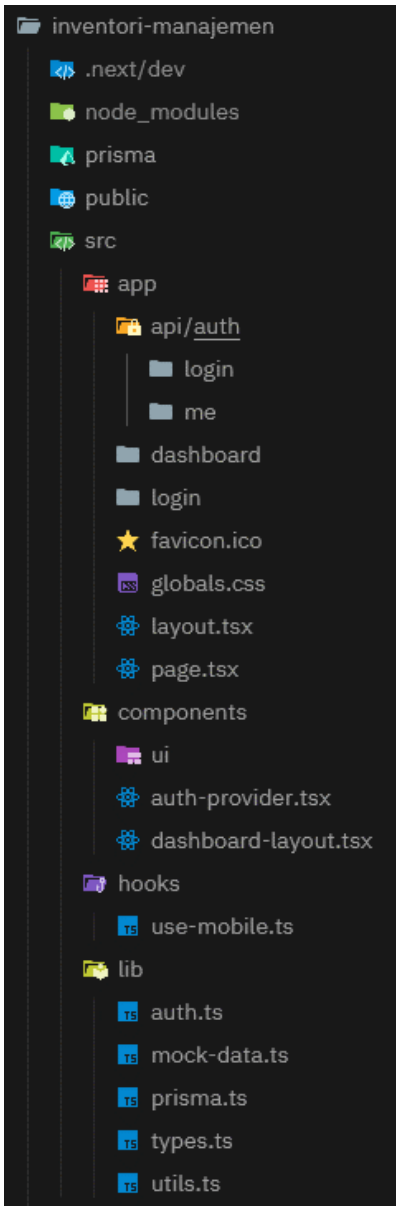
Spesifikasi perangkat lunak dan alat bantu (*tools*) yang digunakan dalam pembangunan 'Sistem Inventori Barang' dirangkum dalam Tabel 5.1 berikut:

Table 3 Spesifkasi perangkat lunak dan alat bantu

Kategori	**Nama Perangkat Lunak / Tools**	**Keterangan**
Sistem Operasi	Windows 11 (64-bit) / Linux Ubuntu (via WSL2)	Platform utama sistem operasi tempat pengembangan dilakukan.
Bahasa Pemrograman	TypeScript	Bahasa pemrograman utama yang digunakan untuk logika *frontend* dan *backend* (superset dari JavaScript).
Framework	Next.js	Kerangka kerja berbasis React untuk pengembangan aplikasi web *fullstack*.
Runtime Environment	Node.js	Lingkungan eksekusi untuk menjalankan JavaScript/TypeScript di sisi server.
Basis Data	PostgreSQL	Sistem manajemen basis data relasional (RDBMS) untuk penyimpanan data persisten.
Code Editor / IDE	Visual Studio Code	Editor kode sumber yang dilengkapi dengan ekstensi pendukung TypeScript dan Git.
Version Control	Git	Sistem pengontrol versi untuk manajemen kode sumber dan kolaborasi.
Web Browser	Google Chrome / Microsoft Edge	Peramban web yang digunakan untuk *debugging* dan pengujian tampilan antarmuka pengguna.

5.3 Struktur Direktori Aplikasi

Struktur direktori 'Sistem Inventori Barang' disusun mengikuti standar arsitektur *Next.js App Router*. Pengorganisasian berkas dilakukan secara hierarkis di dalam direktori src untuk memisahkan logika bisnis, tampilan antarmuka, dan konfigurasi sistem. Pendekatan ini memudahkan pemeliharaan kode (*maintainability*) dan kolaborasi pengembangan.
Berikut adalah visualisasi pohon direktori utama proyek:



Gambar 12 Struktur Folder

Penjelasan Komponen Direktori Utama:

1. **src/app (App Router)** Direktori ini merupakan inti dari routing aplikasi. Next.js menggunakan sistem *file-system based routing*, di mana setiap folder merepresentasikan segmen URL.
    - Folder seperti /dashboard dan /login secara otomatis menjadi rute yang dapat diakses melalui browser.
    - Berkas layout.tsx mendefinisikan kerangka antarmuka yang konsisten antar halaman, sedangkan page.tsx berisi konten spesifik halaman tersebut.

2. **src/app/api (Server-Side Logic)** Direktori ini berfungsi sebagai *Route Handlers* yang menangani permintaan HTTP (*request/response*) di sisi server. Dalam sistem ini, folder api/auth secara spesifik menangani logika *backend* untuk proses otentikasi pengguna dan manajemen sesi tanpa perlu membuat server terpisah.

3. **src/components (Reusable UI)** Direktori ini menyimpan elemen-elemen antarmuka modular yang memisahkan lapisan tampilan (*view*) dari logika halaman.
    - Folder ui berisi komponen atomik (tombol, input, kartu).
    - Komponen seperti dashboard-layout.tsx digunakan untuk menyusun struktur navigasi yang kompleks namun tetap rapi.

4. **src/lib (Utilities & Configuration)** Direktori ini berisi fungsi-fungsi utilitas global dan konfigurasi pihak ketiga. Berkas krusial di dalamnya meliputi:
    - **prisma.ts**: Berfungsi untuk menginisialisasi *Prisma Client* sebagai *singleton*, memastikan hanya ada satu koneksi aktif ke basis data PostgreSQL untuk mencegah kebocoran koneksi (*connection leak*).
    - **auth.ts**: Menyimpan konfigurasi logika otentikasi, strategi sesi, dan proteksi rute (*route protection*).

5. **prisma/ (Database Schema)** Direktori ini terletak di akar proyek (di luar src) dan dikhususkan untuk menyimpan berkas schema.prisma. Berkas ini mendefinisikan model data (Tabel User, Barang, Transaksi) serta menyimpan berkas migrasi yang diperlukan untuk mensinkronisasi struktur basis data dengan kode program.

Kesimpulan Arsitektur Struktur direktori ini menerapkan prinsip **Modularitas** yang ketat, di mana setiap fitur memiliki ruang lingkup yang jelas. Selain itu, arsitektur ini memfasilitasi pemisahan yang efisien antara *Client Components* (interaktif) dan *Server Components* (pemrosesan data), sehingga menghasilkan performa aplikasi web yang optimal dan waktu muat (*load time*) yang cepat.

5.4 Implementasi Fitur Utama

Implementasi fitur otentikasi merupakan komponen krusial yang berfungsi sebagai gerbang keamanan utama sistem. Fitur ini menjamin bahwa hanya pengguna yang terdaftar dan memiliki hak akses valid yang dapat memasuki lingkungan sistem. Dalam pengembangannya menggunakan Next.js dan Prisma ORM, alur logika otentikasi dirancang melalui beberapa tahapan pemrosesan data yang aman dan sistematis.

Berikut adalah uraian algoritma dan logika yang terjadi di balik layar saat proses *login* berlangsung:

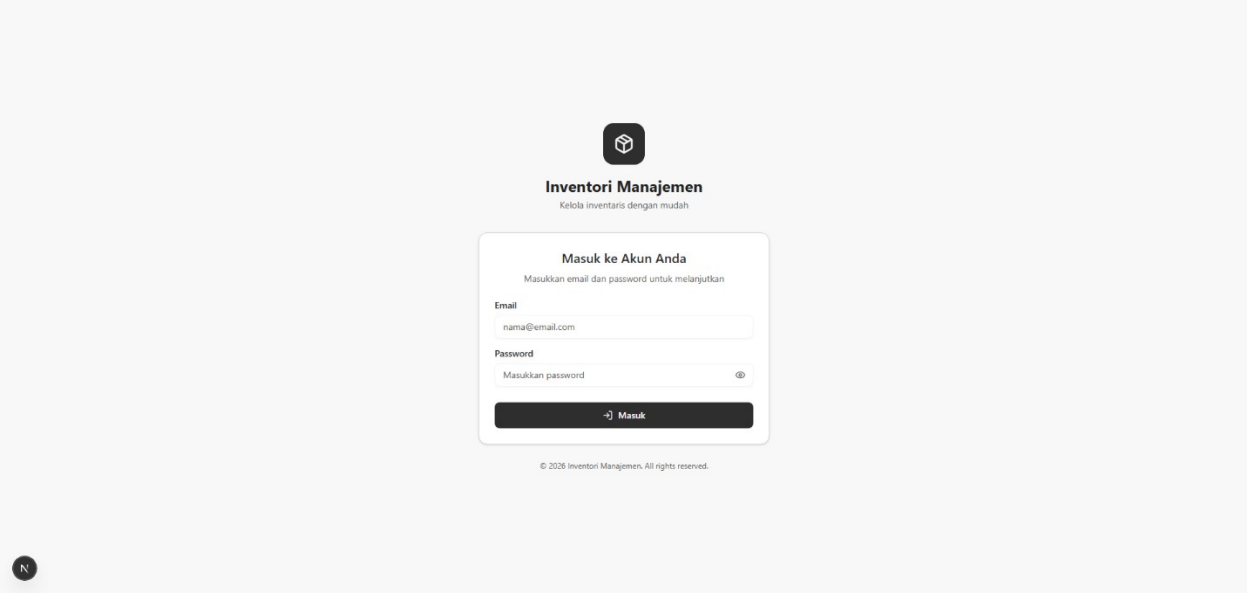
1. **Penerimaan Input Kredensial (*Input Reception*)** Proses dimulai pada sisi antarmuka (*client-side*) ketika pengguna memasukkan informasi kredensial berupa *username* dan *password* pada formulir login. Sistem memvalidasi kelengkapan input secara visual sebelum data diproses lebih lanjut. Setelah pengguna menekan tombol masuk, aplikasi mengemas data tersebut ke dalam format JSON (*JavaScript Object Notation*) dan mengirimkannya ke *server endpoint* menggunakan metode HTTP POST. Penggunaan metode POST memastikan bahwa data sensitif dikirimkan di dalam *body request*, bukan melalui URL, guna menjaga kerahasiaan informasi selama transmisi data.

2. **Validasi dan Pencarian Akun (*Account Lookup*)** Di sisi server (*server-side*), logika bisnis menerima paket data tersebut dan segera memprosesnya. Sistem memanfaatkan *Prisma ORM* sebagai jembatan komunikasi ke basis data PostgreSQL. Prisma menjalankan fungsi kueri asinkron (*asynchronous query*) untuk memindai tabel pengguna (*User Table*). Algoritma pencarian bekerja dengan mencocokkan *username* yang diinputkan pengguna dengan kolom *username* yang ada di basis data. Jika *username* tidak ditemukan, sistem akan segera menghentikan proses dan mengembalikan respon kegagalan untuk efisiensi sumber daya.

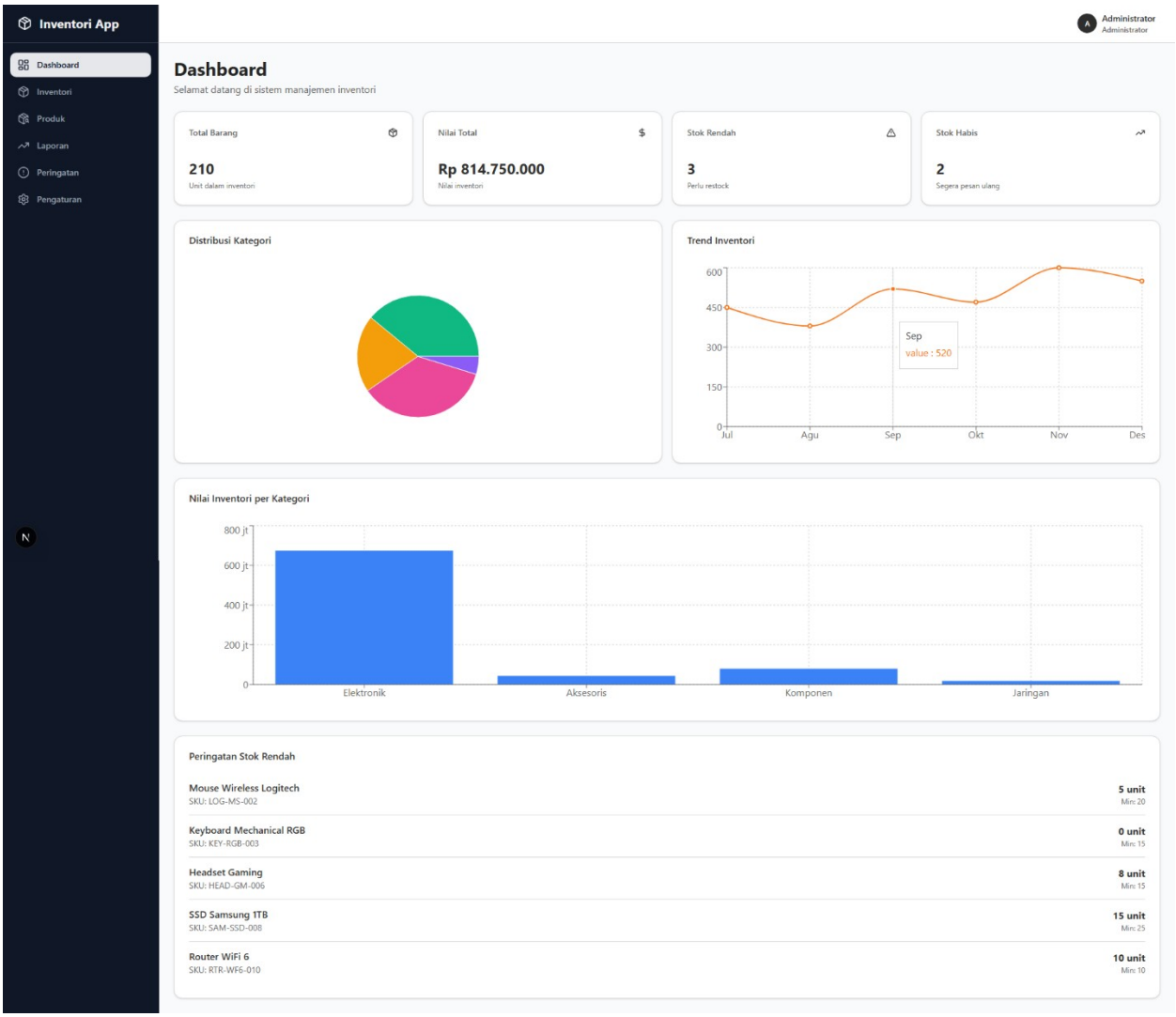
3. **Verifikasi Keamanan (*Secure Verification*)** Apabila *username* ditemukan, sistem tidak serta-merta memberikan akses. Langkah selanjutnya adalah verifikasi kata sandi. Mengikuti standar keamanan industri, sistem **tidak pernah** menyimpan atau membandingkan kata sandi dalam bentuk teks asli (*plain text*). Sistem melakukan komparasi kriptografi antara kata sandi yang baru diinputkan dengan *hash* kata sandi yang tersimpan di basis data. Proses ini melibatkan pustaka keamanan (seperti bcrypt atau argon2) yang melakukan proses *hashing* ulang terhadap input pengguna dan membandingkan string *hash* yang dihasilkan. Mekanisme ini memastikan bahwa meskipun basis data bocor, kata sandi asli pengguna tetap terlindungi dan sulit diretas.

4. **Penanganan Hasil (*Response Handling*)** Berdasarkan hasil verifikasi langkah sebelumnya, sistem mengeksekusi logika percabangan akhir:
    - **Jika Data Cocok (Valid):** Sistem menyatakan otentikasi berhasil. Server kemudian menginisialisasi sesi pengguna (*session*) atau menerbitkan token akses aman. Pengguna kemudian diarahkan (*redirect*) secara otomatis meninggalkan halaman login menuju halaman utama *Dashboard* sesuai dengan hak akses yang dimilikinya.
    - **Jika Data Tidak Cocok (Invalid):** Sistem menolak permintaan akses. Server mengirimkan respon kesalahan (*error response*) kembali ke antarmuka pengguna. Pesan umpan balik ditampilkan pada formulir login untuk memberitahu pengguna bahwa kombinasi *username* atau *password* yang dimasukkan salah, tanpa memberikan detail spesifik mana yang salah demi mencegah serangan *brute-force*.

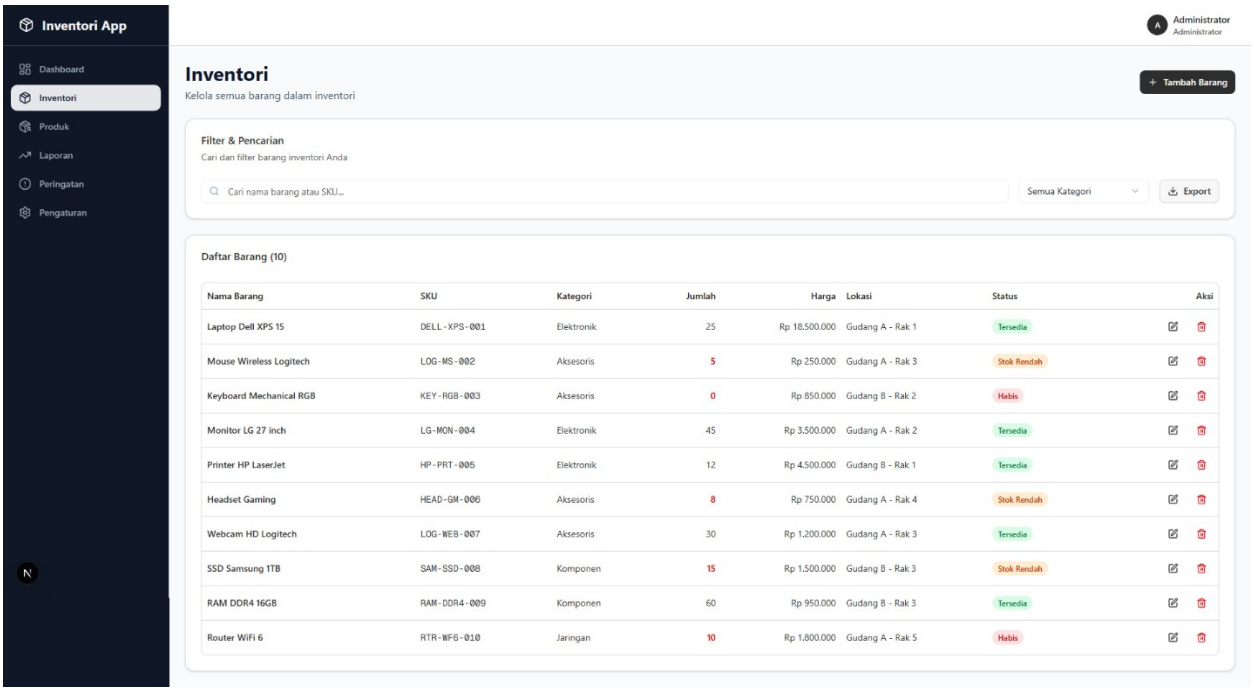
5. Tampilan UI



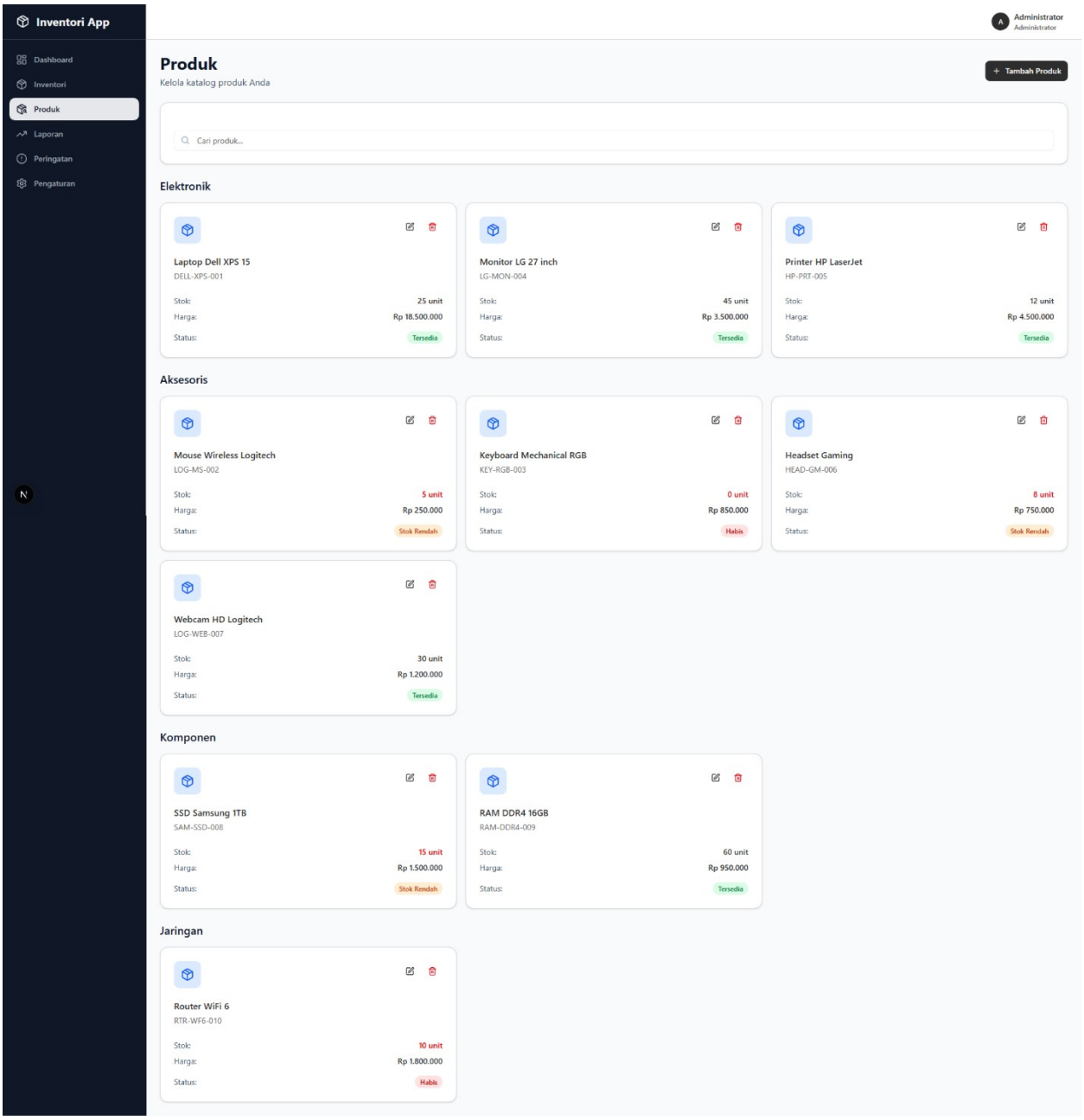
Gambar 13 Halaman Login



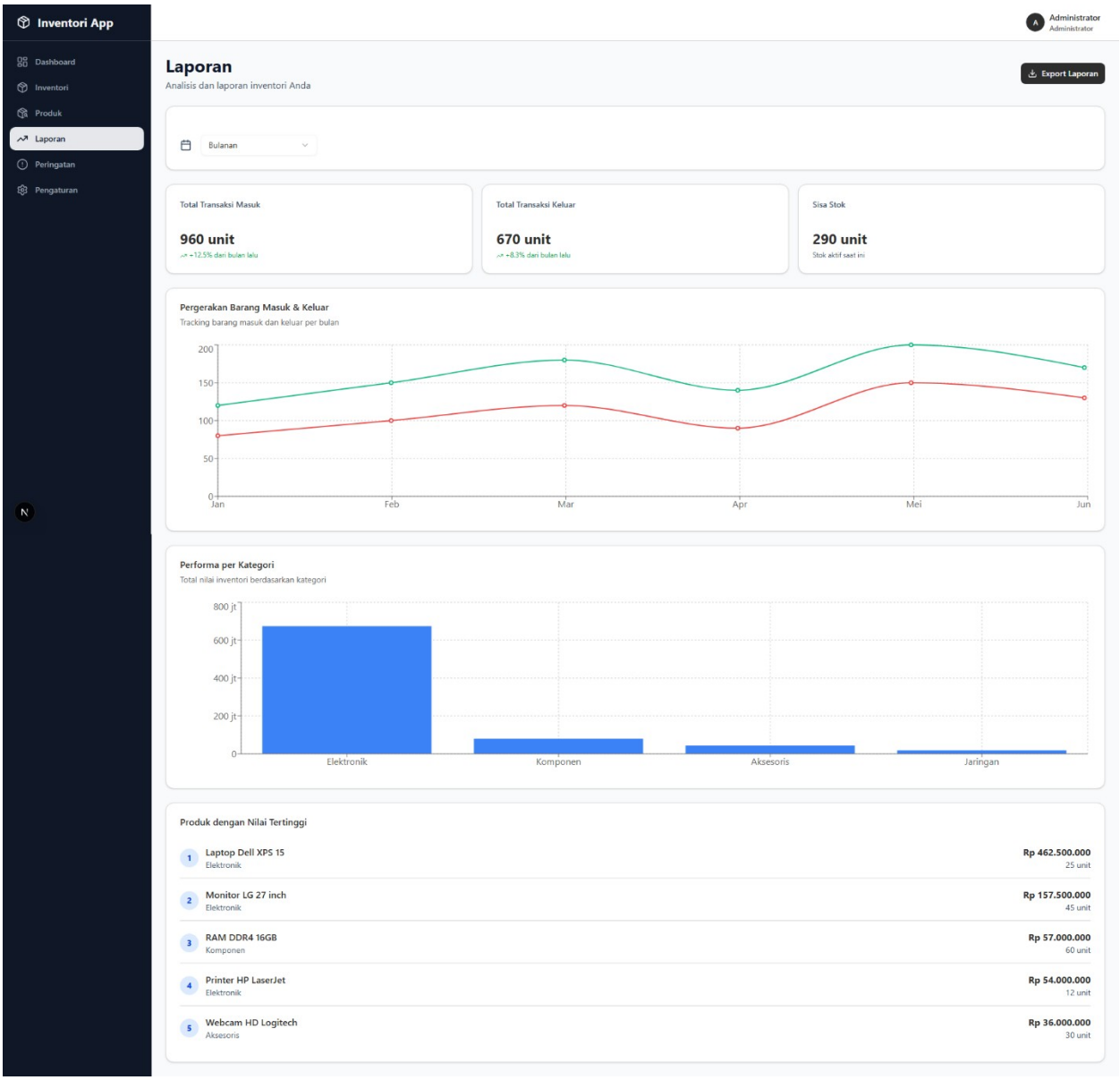
Gambar 14 Halaman dashboard



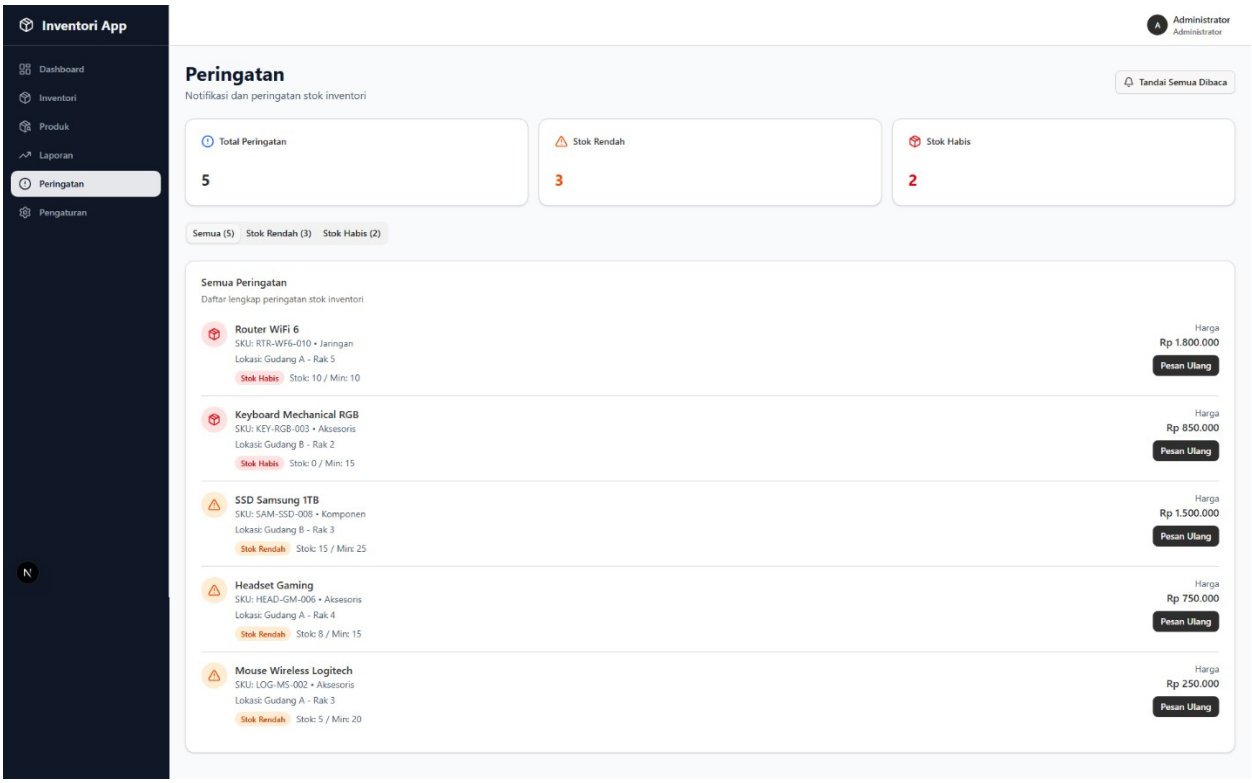
Gambar 15 Halaman inventori



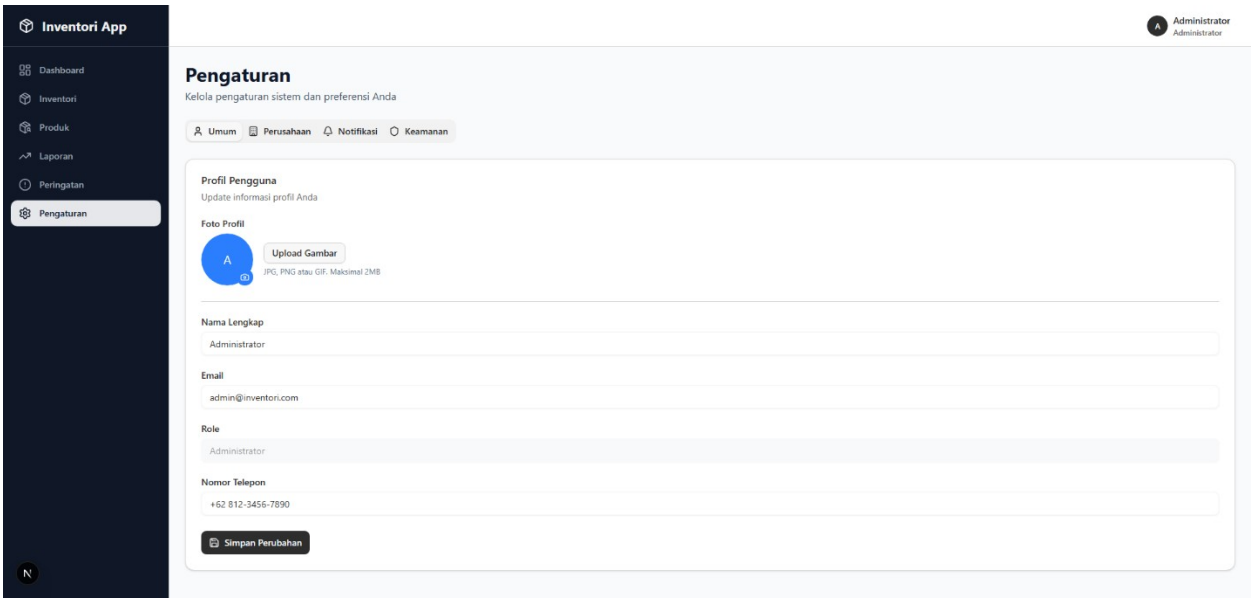
Gambar 16 Halaman produk



Gambar 17 Halaman laporan



Gambar 18 Halaman peringatan



Gambar 19 Halaman pengaturan

5.5 Keamanan dan Kontrol Akses

Dalam tahap implementasi, aspek keamanan menjadi prioritas utama untuk menjamin integritas data dan melindungi privasi pengguna. Sistem Inventori Barang menerapkan pendekatan keamanan berlapis (*layered security approach*) yang mencakup perlindungan data sensitif pada tingkat basis data dan pembatasan hak akses pada tingkat aplikasi. Mekanisme teknis yang diterapkan diuraikan sebagai berikut:

5.5.1 Pengamanan Kredensial (*Password Hashing*)

Sistem menerapkan standar keamanan data yang ketat terkait penyimpanan kredensial pengguna. Kata sandi (*password*) pengguna **tidak pernah** disimpan dalam format teks biasa (*plain text*) di dalam basis data. Penyimpanan dalam bentuk teks asli dinilai sangat berisiko karena dapat mengekspos seluruh akun pengguna apabila basis data berhasil diretas oleh pihak yang tidak bertanggung jawab.

Sebagai langkah mitigasi, sistem menggunakan algoritma *hashing* kriptografi satu arah (seperti *bcrypt*). Mekanisme ini bekerja dengan mengubah kata sandi yang diinputkan pengguna menjadi serangkaian karakter acak (*string hash*) yang panjang dan kompleks sebelum disimpan ke tabel pengguna. Sifat algoritma ini adalah *irreversible* (tidak dapat dikembalikan ke bentuk asalnya), sehingga kerahasiaan kata sandi asli tetap terjamin meskipun penyerang mendapatkan akses fisik ke file basis data.

5.5.2 Otorisasi Berbasis Peran (*Role-Based Access Control*)

Selain keamanan data, sistem juga mengimplementasikan mekanisme otorisasi yang ketat menggunakan metode *Role-Based Access Control* (RBAC). Metode ini berfungsi untuk mengatur dan membatasi hak akses pengguna berdasarkan peran (*role*) yang melekat pada akun mereka, yaitu sebagai **Admin** atau **Petugas Gudang (User)**.
Secara teknis, logika sistem (melalui *middleware* atau *server-side logic*) akan melakukan pemeriksaan atribut 'Role' pada sesi pengguna setiap kali terdapat permintaan akses ke halaman atau fitur tertentu.

- **Hak Akses Admin:** Akun dengan peran 'Admin' diberikan privilese penuh (*full privileges*), termasuk akses ke halaman sensitif seperti 'Kelola Pengguna' untuk manajemen akun staf.
- **Hak Akses Petugas Gudang:** Akun dengan peran 'User' dibatasi secara ketat (*restricted access*). Petugas Gudang hanya diizinkan mengakses fitur operasional seperti 'Input Transaksi' dan 'Lihat Stok'.

Jika Petugas Gudang mencoba mengakses URL atau fitur yang dikhususkan untuk Admin, sistem akan secara otomatis memblokir permintaan tersebut dan mengarahkan pengguna kembali

ke halaman yang diizinkan. Pemisahan wewenang ini krusial untuk mencegah eskalasi hak akses (*privilege escalation*) dan meminimalkan risiko kesalahan manusia atau penyalahgunaan data dalam operasional sistem.

BAB VI
PENGUJIAN SISTEM

6.1 Pengujian *Black Box*

Tahap pengujian merupakan fase validasi akhir untuk memastikan perangkat lunak yang dibangun memiliki kualitas yang baik dan memenuhi standar yang ditetapkan. Metode yang digunakan dalam pengujian Sistem Inventori Barang ini adalah *Black Box Testing*. Metode ini dipilih karena efektivitasnya dalam mengevaluasi kinerja sistem dari kacamata pengguna tanpa intervensi teknis yang mendalam terhadap kode program.

Berikut adalah rincian metodologi pengujian yang diterapkan:

- **Definisi dan Konsep Dasar** *Black Box Testing* adalah metode pengujian perangkat lunak yang berfokus sepenuhnya pada persyaratan fungsional aplikasi. Dalam metode ini, penguji memperlakukan sistem sebagai sebuah "kotak hitam" tertutup. Penguji tidak memiliki pengetahuan tentang struktur kode internal (*internal code structure*), logika algoritma, maupun jalur pemrograman yang membangun sistem tersebut. Fokus utama pengujian hanyalah pada hubungan antara *Input* yang dimasukkan ke dalam sistem dan *Output* yang dihasilkan. Jika *output* yang keluar sesuai dengan yang diharapkan berdasarkan *input* yang diberikan, maka fungsi tersebut dianggap valid.

- **Sudut Pandang Pengguna (*End-User Perspective*)** Pengujian ini dilakukan dengan mensimulasikan perilaku pengguna akhir (*end-user*). Penguji berinteraksi dengan antarmuka pengguna (*User Interface*) selayaknya seorang petugas gudang atau administrator yang sedang bekerja. Pendekatan ini sangat krusial untuk memastikan bahwa alur navigasi, responsivitas sistem, dan pesan umpan balik (*feedback*) terasa natural, intuitif, dan mudah dipahami. Hal ini menjamin bahwa sistem tidak hanya berfungsi secara teknis, tetapi juga ergonomis saat digunakan dalam operasional nyata.

- **Tujuan Validasi** Tujuan utama pelaksanaan *Black Box Testing* ini adalah untuk memverifikasi kesesuaian implementasi sistem dengan spesifikasi **Kebutuhan Fungsional** yang telah didefinisikan pada Bab III. Secara khusus, pengujian ini difokuskan pada validasi fitur **Otentikasi (Login)**. Pengujian bertujuan untuk membuktikan bahwa mekanisme keamanan sistem mampu menolak akses yang tidak sah dan memberikan akses

kepada pengguna yang valid tanpa adanya kesalahan fungsi (*functional errors*) atau kegagalan sistem (*system crash*) saat menerima berbagai variasi data masukan.

6.2 Skenario Pengujian

Berdasarkan metodologi *Black Box Testing*, disusunlah serangkaian skenario uji coba untuk fitur Otentikasi (*Login*). Tabel berikut merincikan langkah-langkah pengujian, data masukan yang digunakan, serta respon sistem yang diharapkan (*Expected Result*) sebagai tolak ukur keberhasilan fungsi.

Table 4 Pengujian

No	Skenario Pengujian	Data Input (Contoh)	Hasil yang Diharapkan
1	**Verifikasi Login Berhasil** Menguji respon sistem saat pengguna memasukkan kombinasi kredensial yang valid dan terdaftar di database.	email: admin@inventori.com/ staff@inventori.com Password: admin123/ staff123 (Valid)	Sistem memvalidasi kecocokan data, membuat sesi pengguna aktif (*session created*), dan secara otomatis mengarahkan (*redirect*) pengguna dari halaman Login ke halaman utama Dashboard.
2	**Verifikasi Login Gagal (Password Salah)** Menguji keamanan sistem saat pengguna memasukkan username benar namun password salah.	Username: admin@inventori.com Password: salah123 (Invalid)	Sistem menolak akses masuk. Halaman tidak berpindah (tetap di /login) dan menampilkan pesan kesalahan (*error alert*) yang informatif: "Username atau Password salah".

No	Skenario Pengujian	Data Input (Contoh)	Hasil yang Diharapkan
3	**Verifikasi Login Gagal (User Tidak Ditemukan)** Menguji respon sistem saat pengguna mencoba masuk dengan username yang belum terdaftar.	Username: user_tidak_ada Password: sembarang	Sistem menolak akses masuk. Menampilkan pesan kesalahan standar untuk keamanan, mencegah pengguna mengetahui apakah username tersebut eksis atau tidak.
4	**Verifikasi Validasi Input Kosong** Menguji mekanisme validasi formulir saat pengguna menekan tombol login tanpa mengisi data.	Username: [Kosong] Password: [Kosong]	Sistem mencegah pengiriman data ke server (*prevent submit*). Menampilkan pesan peringatan visual pada antarmuka (UI) bahwa "Kolom username dan password wajib diisi".
5	**Profil user**	Klik profil user pada kanan atas halaman	Ketika klik profil ke halaman profil, Ketika klik pengaturan ke halaman pengaturan, Ketika klik keluar akan terlogout dari website
5	**Akses Semua Halaman**	Klik per halaman	Halaman memiliki tampilan masing masing
6	**Filter Kategori di Halaman Inventori**	Klik filter	Produk terfilter sesuai dengan kategori
7	**Tombol Tambah Barang dan Aksi Halaman Inventori**	Klik tombol tambah barang dan aksi (edit atau delete)	Menampilkan modal dialog tambah barang dan edit atau delete
8	**Tombol Tambah Produk dan Aksi**	Klik tombol tambah produk dan aksi (edit atau	Menampilkan modal dialog tambah produk dan edit atau

No	Skenario Pengujian	Data Input (Contoh)	Hasil yang Diharapkan
	Halaman Produk	delete)	delete
9	**Upload gambar**	Klik halaman pengaturan lalu upload gambar	Bisa mengupload gambar

BAB VII
PEMELIHARAAN SISTEM

Pemeliharaan sistem merupakan tahapan lanjutan setelah proses implementasi dan pengujian perangkat lunak selesai dilakukan. Tahap ini memiliki peranan yang sangat penting untuk memastikan bahwa Sistem Inventori Barang berbasis web dapat terus beroperasi secara optimal, stabil, dan sesuai dengan kebutuhan pengguna dalam jangka panjang. Seiring dengan penggunaan sistem secara nyata, berbagai perubahan, baik yang bersumber dari kesalahan internal sistem maupun faktor eksternal, dapat memengaruhi kinerja dan keandalan sistem. Oleh karena itu, diperlukan strategi pemeliharaan yang terencana dan berkelanjutan agar sistem tetap mampu mendukung proses pengelolaan inventori secara efektif dan efisien.

7.1 Pemeliharaan Korektif (*Corrective Maintenance*)

Pemeliharaan korektif merupakan aktivitas pemeliharaan yang dilakukan untuk memperbaiki kesalahan atau cacat (*bug*) yang ditemukan setelah sistem inventori barang berbasis web diimplementasikan dan digunakan oleh pengguna secara nyata. Kesalahan tersebut dapat muncul akibat kondisi yang tidak terdeteksi pada tahap pengujian.

Contoh Implementasi:

- **Kasus (Bug Validasi Stok):** Ditemukan celah di mana Petugas Gudang masih bisa menginput jumlah barang keluar melebihi stok yang tersedia jika koneksi internet lambat (terjadi *delay* validasi), menyebabkan stok di *database* menjadi minus (negatif).
- **Tindakan Perbaikan:** Pengembang memperbaiki kode *backend* (API) dengan menambahkan validasi ganda di sisi server menggunakan *database transaction* Prisma, sehingga sistem akan otomatis menolak transaksi jika stok akhir terhitung kurang dari nol.

7.2 Pemeliharaan Adaptif (*Adaptive Maintenance*)

Pemeliharaan adaptif dilakukan sebagai respons terhadap perubahan lingkungan eksternal yang memengaruhi operasional sistem. Lingkungan eksternal tersebut meliputi pembaruan perangkat keras, perubahan sistem operasi, pembaruan web server, maupun perubahan versi basis data.

Contoh Implementasi:

- **Kasus (Pembaruan Lingkungan Server):** Penyedia layanan *hosting* melakukan pembaruan keamanan dan mewajibkan penggunaan **Node.js versi 20 (LTS)**, sementara aplikasi saat ini dibangun di atas versi 18.
- **Tindakan Penyesuaian:** Tim pengembang melakukan pembaruan konfigurasi pada file package.json, memperbarui *library* yang usang, dan melakukan pengujian ulang (*re-testing*) untuk memastikan aplikasi Next.js tetap berjalan stabil di lingkungan *runtime* Node.js yang baru.

7.3 Pemeliharaan Preventif (*Preventive Maintenance*)

Pemeliharaan preventif merupakan upaya pemeliharaan yang dilakukan secara berkala dengan tujuan mencegah terjadinya gangguan atau kerusakan sistem di masa mendatang. Fokus utamanya adalah menjaga performa, keamanan, dan keandalan sistem sebelum masalah terjadi.

Contoh Implementasi:

- **Aktivitas (Pencadangan Data Otomatis):** Menerapkan mekanisme **Auto-Backup Database** setiap hari pada pukul 00:00 WIB. Sistem dikonfigurasi untuk menjalankan perintah pg_dump pada database PostgreSQL dan mengirimkan file cadangan (*backup file*) ke penyimpanan awan (*cloud storage*) terpisah untuk mengamankan data dari risiko kegagalan server fisik.

7.4 Pemeliharaan Sempurna (*Perfective Maintenance*)

Pemeliharaan sempurna atau *perfective maintenance* adalah pemeliharaan yang dilakukan untuk meningkatkan kualitas sistem berdasarkan umpan balik pengguna dan kebutuhan yang berkembang. Pemeliharaan ini berfokus pada penyempurnaan fungsi dan peningkatan kenyamanan pengguna.

Contoh Implementasi:

- **Kasus (Penambahan Fitur Ekspor):** Admin membutuhkan data laporan stok dalam format Excel agar bisa diolah lebih lanjut untuk keperluan rapat manajemen, karena fitur cetak PDF saat ini dianggap kurang fleksibel.
- **Tindakan Penyempurnaan:** Pengembang menambahkan tombol fitur **"Export to Excel"** pada halaman Laporan, yang memungkinkan Admin mengunduh data rekapitulasi stok dan transaksi ke dalam format .xlsx secara instan.

KESIMPULAN

Sistem Inventori Barang berbasis web ini telah berhasil sebagai solusi terkomputerisasi untuk mengotomatisasi pengelolaan stok internal dan pencatatan transaksi barang masuk serta keluar secara real-time. Melalui implementasi teknologi modern seperti Next.js dan PostgreSQL, sistem ini mampu menyediakan informasi stok yang terpusat, konsisten, dan akurat. Fokus utama pengembangan telah tercapai, yakni meningkatkan efisiensi operasional gudang dengan adanya mekanisme validasi stok otomatis untuk mencegah kesalahan input serta fitur pelaporan yang memudahkan pemantauan mutasi barang.

Dari sisi keamanan dan kualitas, sistem ini telah menerapkan standar industri dengan menggunakan pengamanan kredensial melalui password hashing dan pembatasan hak akses berbasis peran (Role-Based Access Control) antara Admin dan Petugas Gudang. Pengujian yang dilakukan dengan metode Black Box membuktikan bahwa seluruh fungsi utama, mulai dari proses otentikasi hingga manajemen data, telah berjalan sesuai dengan spesifikasi kebutuhan yang ditetapkan. Dengan demikian, sistem ini telah memenuhi kriteria kesesuaian fungsional dan kebergunaan, sehingga siap digunakan untuk mendukung aktivitas inventori organisasi secara berkelanjutan melalui strategi pemeliharaan yang terencana.